

Stereo Sum Embedding Report

Team 03

Team Members

Khristian Neal - NBT292

Ian Rohan - USM404

Quinn Westerberg - HFS654

Steganography 4463 - Summer 2025

August 7th, 2025

Team Contributions

Khristian Neal (nbt292) - Provided sample wav files for embedding, helped to test program, aided in writing the report, project leader

Ian Rohan (usm404) - Responsible for coding program, provided sample wav file for embedding, provided text/bmp/png files for embedding into wav, tested program, aided in writing the report

Quinn Westerberg (hfs654) - Provided sample wav files for embedding, helped to test program, created block diagram, aided in writing the report

Project Introduction

Explanation and Approach

Our steganography project involves a stereo audio steganography tool that embeds and extracts various data file types - such as text, picture (.bmp and .png) and sound (.wav) - in raw 16-bit stereo WAV format. This program employs a sum-based embedding method that alters the sum of the left-right stereo sample pairs in configurable blocks, enabling a balance between data capacity and audio transparency.

While different from classical bit-by-bit approaches, stereo sum embedding is an alternative form of LSB embedding. It operates by dividing the stereo samples into blocks (for example, 2, 4, 6, 8 samples), summing their integer values, and then setting the least significant bits of the sum to equal the message bits. If the desired bits are already there, the block is not changed; otherwise, the tool performs minimal adjustments to one or more samples. During these adjustments, the program will attempt to maintain audio quality while keeping all values within range of 16-bit signed integers (-32,768 - 32,767).

The tool supports variable embedding depth, from 1 to 16 bits per block. While it is theoretically possible to embed at higher depths (12 or 16 bits), these settings have a tendency to produce audible distortion, especially in music-intensive WAVs. Experimentally, an embedding depth of 8 bits per block has proved to be the optimal compromise between capacity and imperceptibility and is the recommended setting for practical use. The tool has embedding and extraction modes with support for arbitrary files. We have coded it to allow for verifying audio compatibility (44.1 kHz stereo, 16-bit PCM WAV, for example) and performs sanity checks of message size. To verify recovery success, users can compare SHA256 hashes of the original and extracted files.

Project Goals

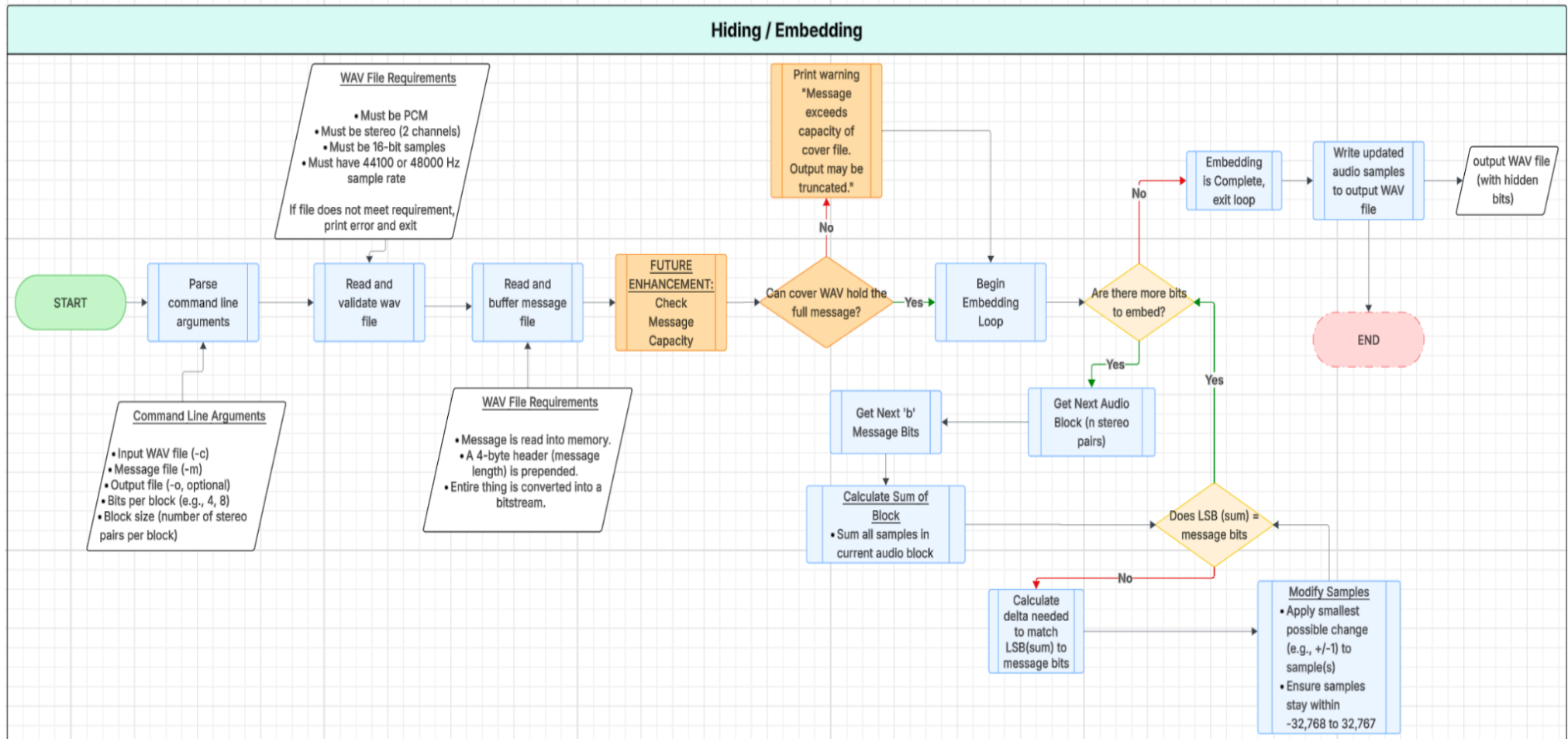
- Implement a fully functional steganography tool with both hiding and extraction capabilities
- Variable embedding parameters (-b bits per block, -n samples per block)
- Maintain perceptual transparency at moderate embedding depths
- Allow hiding of a wide range of file types (tested with .txt, .bmp, .wav)

Summary of Results

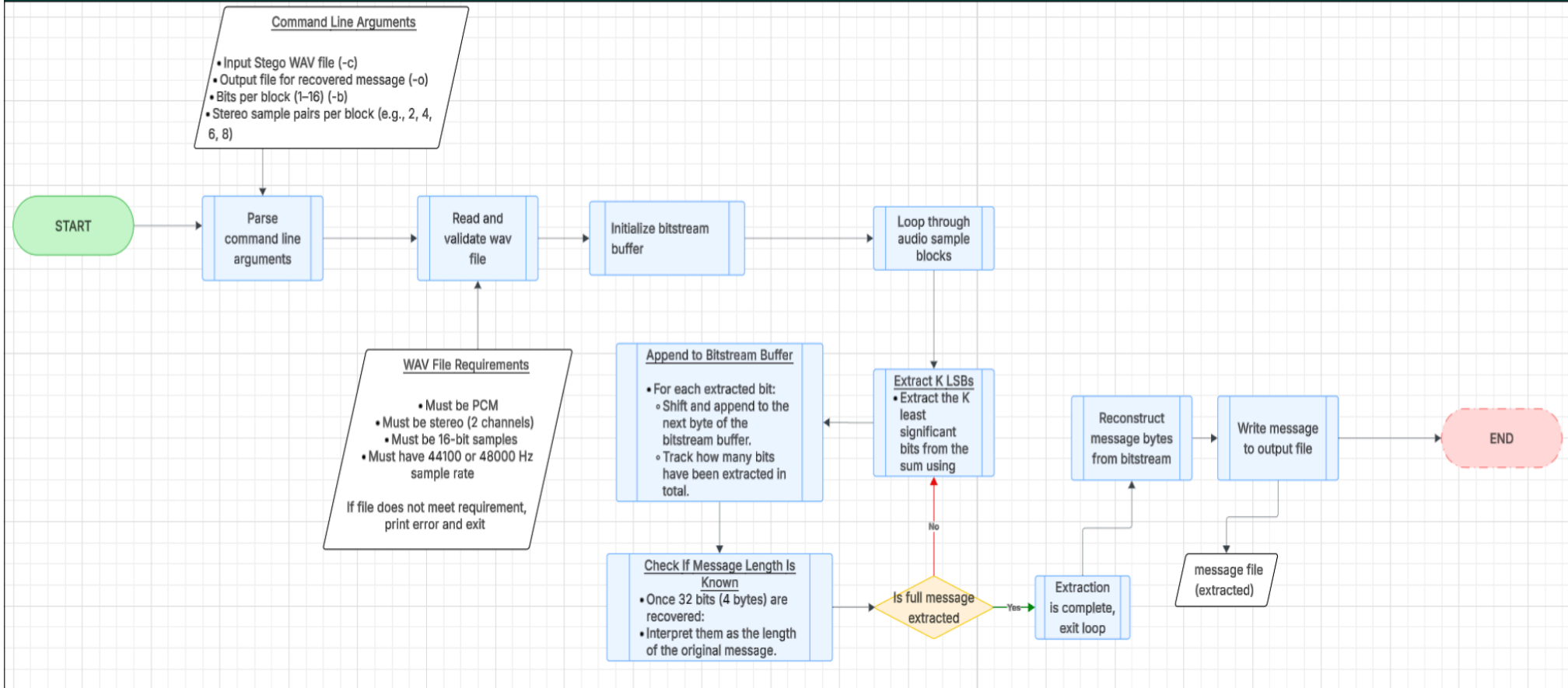
- The tool successfully hides and extracts messages using stereo sample block sums.
- Recovery accuracy is consistently high for .txt and .bmp files.
- .wav file embedding is supported but requires careful tuning of embedding depth to avoid corruption.
- Files embedded at -b 8 -n 4 remain imperceptible to human hearing in most cases.
- At higher embedding rates (e.g., 12–16 bits), audio degradation or incomplete recovery may occur due to sample value limitations or block adjustability.

Overall, this project illustrates the feasibility of sum-based stereo LSB embedding for audio steganography and offers an extensible framework for possible future improvements like encrypted payloads, mono support, or adaptive embedding

Functional Block Diagram



Extracting



Discussion and Overall Hiding/Extracting Techniques

Our steganography tool uses a sum-based stereo embedding scheme, where the blocks of stereo audio samples are modified such that the least significant bits (LSBs) of the block sum convey the desired message bits. This method is conceptually similar to standard LSB embedding but operates at the block level, with better control over distortion and the potential for fine-grained capacity control using tunable parameters. During embedding, message bits are processed in most-significant-bit (MSB) first order within each byte. This ensures that the reconstructed message maintains byte alignment and consistency.

Hiding Technique

Our hiding process consists of the following key steps:

1. WAV Validation:

The original audio file is verified to ensure it's a 16-bit PCM stereo WAV file at a supported sample rate (44.1 kHz or 48 kHz). This ensures uniform data structure and compatibility.

2. Message Preparation:

The message file is loaded into memory and prepended with a 4-byte header containing its total length in bytes. This lets the extractor know when to finish reading.

3. Block Processing:

The audio data is broken up into blocks, each containing n pairs of stereo samples (selectable with `-n`). So, for instance, a block size of $n = 4$ would have 4 left/right sample pairs in the block (8 samples).

4. Sum-Based Encoding:

The sum of the integers of all the samples in the block is calculated. The least significant b bits of that sum (where b is the embedding depth specified by the user) are contrasted with the next b bits of the message.

- If the LSBs of the sum already match the message bits, no changes are made.
- If they don't match, the algorithm adjusts one or more samples in the block so that the new sum has the correct LSBs.
- Adjustments are carefully bound to keep all sample values within the 16-bit signed range ($-32,768$ to $32,767$), and only minimal changes are attempted.

5. Writing the Stego File:

The modified audio data is written back to a new WAV file, preserving the original header structure.

Extraction Technique

The extraction process is essentially the inverse of the embedding process:

1. **WAV Validation and Audio Data Loading:**

The steganographic WAV file is verified and its audio data loaded.

2. **Block Reading and Sum Extraction:**

The audio data is processed in blocks of the same size (n). For each block, the sum of the stereo samples is computed, and the least significant b bits are extracted.

3. **Bit Buffering:**

Extracted bits are stored into a buffer. After collecting 32 bits, the first four bytes are interpreted as the length of the hidden message.

4. **Message Assembly:**

The tool continues reading blocks and extracting bits until the specified number of bytes (plus the 4-byte header) has been reconstructed.

5. **Output Writing:**

If the extracted file is a standard data file (e.g., .txt, .bmp), it is written directly. If it is an embedded .wav, a new WAV header is constructed and prepended to the audio data during output.

Parameter Configuration and Tradeoffs

- **-b <bits per block>:** Controls how many bits are embedded in each block. Larger values increase capacity but also increase the risk of distortion and embedding failure if the sample values cannot be adjusted adequately.
- **-n <samples per block>:** Controls the number of stereo sample pairs per block. Larger blocks smooth out embedding noise but reduce overall capacity.

Strengths of this Technique

- **Imperceptibility:**
At 8 bits per block or lower with moderate block sizes (e.g., -n 4), the audio distortion is typically inaudible.
- **Resilience:**
Sum-based embedding is less predictable than classic per-sample LSB manipulation, making it harder to detect via histogram analysis.

- **Flexibility:**
Users can choose parameters that best suit their use case (capacity vs. quality).

Limitations

- **Block Embedding Failures:**
If the required change in the sample sum to match the desired message bits exceeds what is feasible within 16-bit signed audio constraints, the block may fail to encode data correctly. This issue becomes more common with higher embedding depths (e.g., -b 12 or higher).
- **Capacity Constraints:**
One inherent limitation of the stereo sum embedding approach is its relatively low embedding capacity. Because each block encodes only a few bits, the number of usable blocks (and thus total capacity) is directly tied to the length of the audio file and the block size. This necessitates the use of large cover audio files for hiding even modestly sized messages.
- **Technical Difficulties:**
A real example occurred when embedding a 1-second .wav file (“BurgerKing.wav”) into a 14-second .wav file (“HomestuckPg1.wav”) using -b 8. Upon extraction, the audio of the recovered .wav was cut off early, indicating that not all message data was successfully stored or retrieved. This illustrates the risk of underestimating message size relative to available cover capacity.
- **Distortion at Higher Depths:**
Embedding more than 8 bits per block (-b 12, -b 16, etc.) causes significant distortion and may result in perceptible artifacts. While technically possible, high embedding depths are better suited for demonstrations or stress testing rather than practical use.
- **WAV-in-WAV Embedding Complexity:**
Embedding one WAV file into another is possible but presents challenges due to the size of audio data, the need to generate or interpret valid headers, and the strict requirements of sample alignment and capacity. This is especially prone to errors if the target .wav file is longer than the available room in the cover audio.

Suggestions for Future Work

While the current implementation successfully demonstrates stereo sum-based audio steganography, several enhancements could make the tool more robust, user-friendly, and versatile:

- **Pre-Embedding Capacity Check:**

A common issue during testing was attempting to embed files that were too large for the selected cover audio, resulting in truncated messages. Implementing a pre-check routine that calculates the maximum embeddable payload size based on the number of blocks and selected bit depth would prevent failed or partial embeddings.

- **Graphical User Interface (GUI):**

Although the tool is fully functional as a command-line utility, a simple GUI frontend could enhance usability for non-technical users. Drag-and-drop functionality, parameter sliders, and real-time capacity feedback would streamline the embedding/extraction process for users.

- **Batch Processing Support:**

A feature allowing batch embedding or extraction (e.g., hiding the same message across multiple audio files) could be useful for high-volume or redundant steganographic workflows.

Images/ Examples

During our testing, we embedded various file types—including text, BMP images, and WAV audio—into stereo WAV cover files using different combinations of block size and embedding depth. To evaluate the perceptual transparency of our technique, we analyzed the resulting stego-audio both audibly (listening tests) and visually using histograms of sample differences and byte distributions. In order to ensure the extracted file matches the embedded file, a SHA256 tool was used to verify integrity. If the hashes of the embedded and extracted file do not match, a demonstration of the corrupted extracted file will be shown (if possible).

We categorized distortion into three levels:

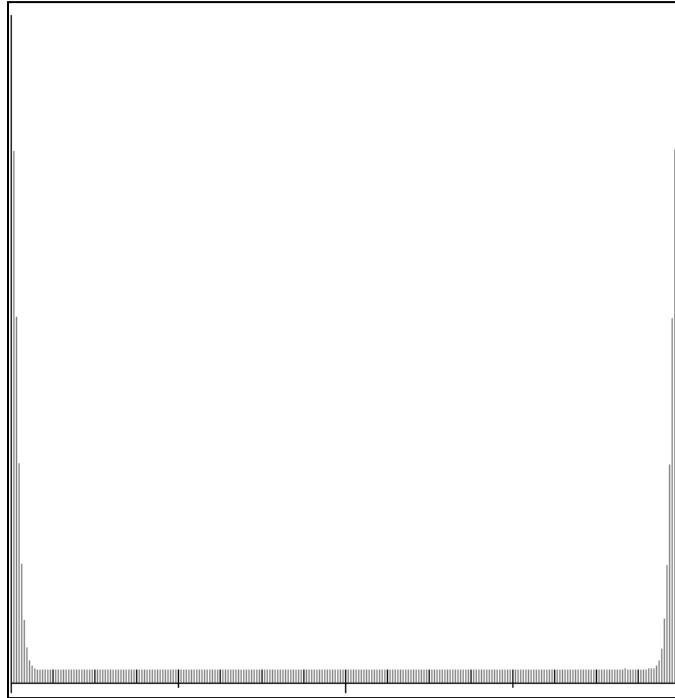
- 1. Not perceptible**
- 2. Perceptible with close attention or comparison**
- 3. Obvious distortion**

To support these observations, we included histogram images and notes for several embedding configurations, alongside a summary chart that visualizes the tradeoff between embedding capacity and perceptibility. To see more examples of our testing, check the “2025_06_CS4463_Team_03_Project_M2.zip” submission for Milestone 2 in which we tested several different file embeds, command line options and audio samples to embed in.

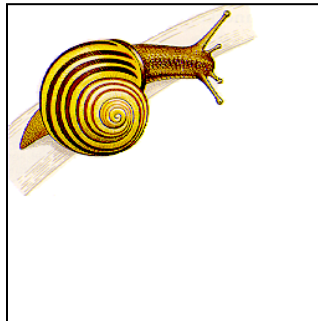
Embedding a BMP into Classical Music

In this test, we embedded a BMP image into a classical music WAV file using 2 samples per block and varying the number of bits per block (4, 8, 12, 16). The WAV file was 3.15 MB in size with a duration of 31 seconds.

For reference, the histogram below (hist_MaidofOrleans.wav_@0x0_z_001.bmp) shows the sample distribution of the original (clean) audio prior to embedding.

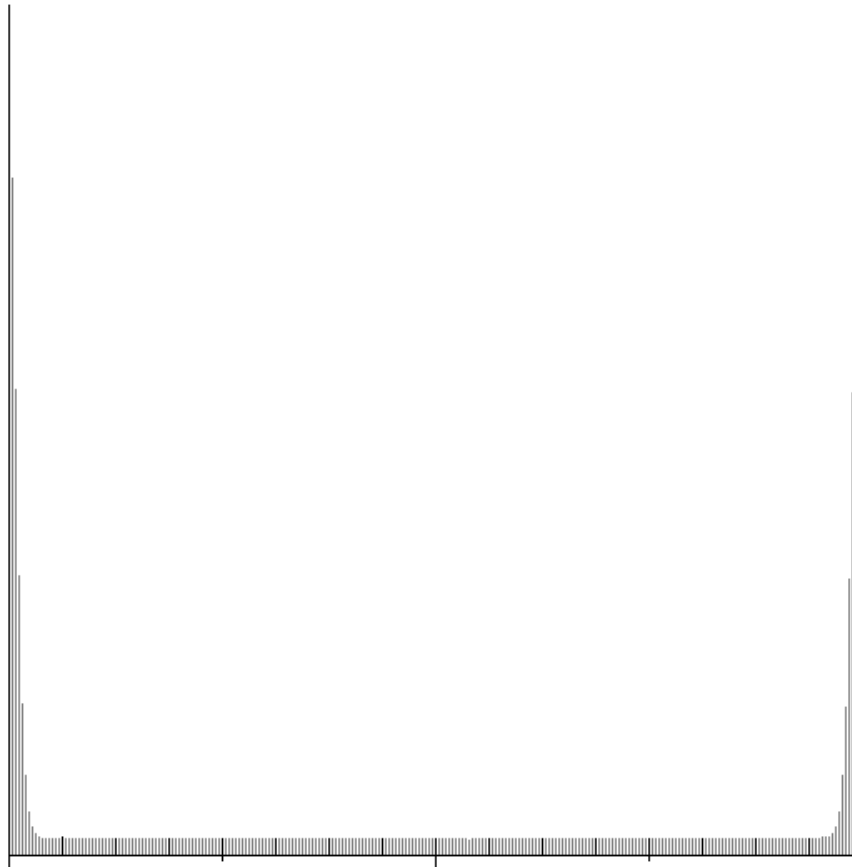


The bitmap image we attempted to embed was snail.bmp, referenced below.



Embedding at 4 Bits Per Block

Histogram of WAV with hidden data:

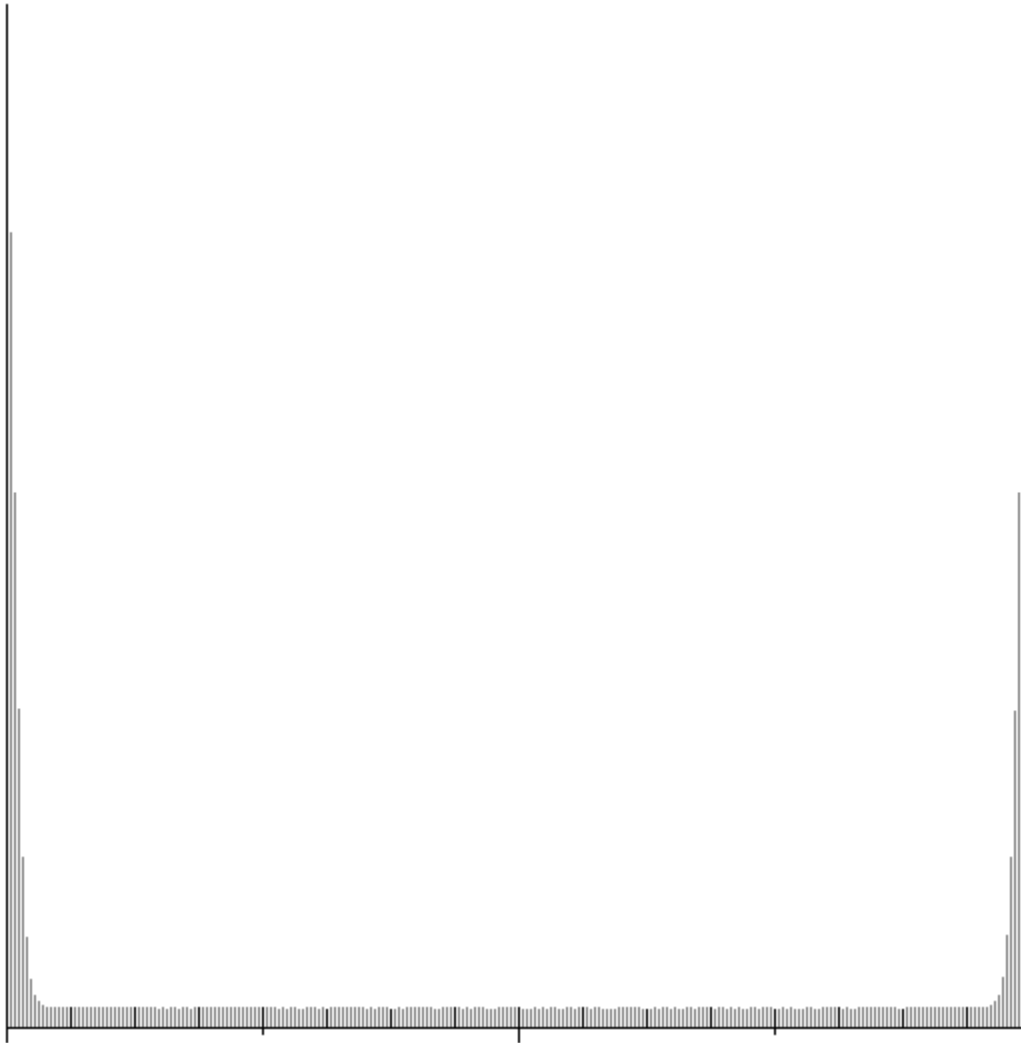


Observations:

- No noticeable distortions in audio (1)
- No corruption from extracted message (SHA256 hashes matched)

Embedding at 8 Bits Per Block

Histogram of WAV with hidden data:

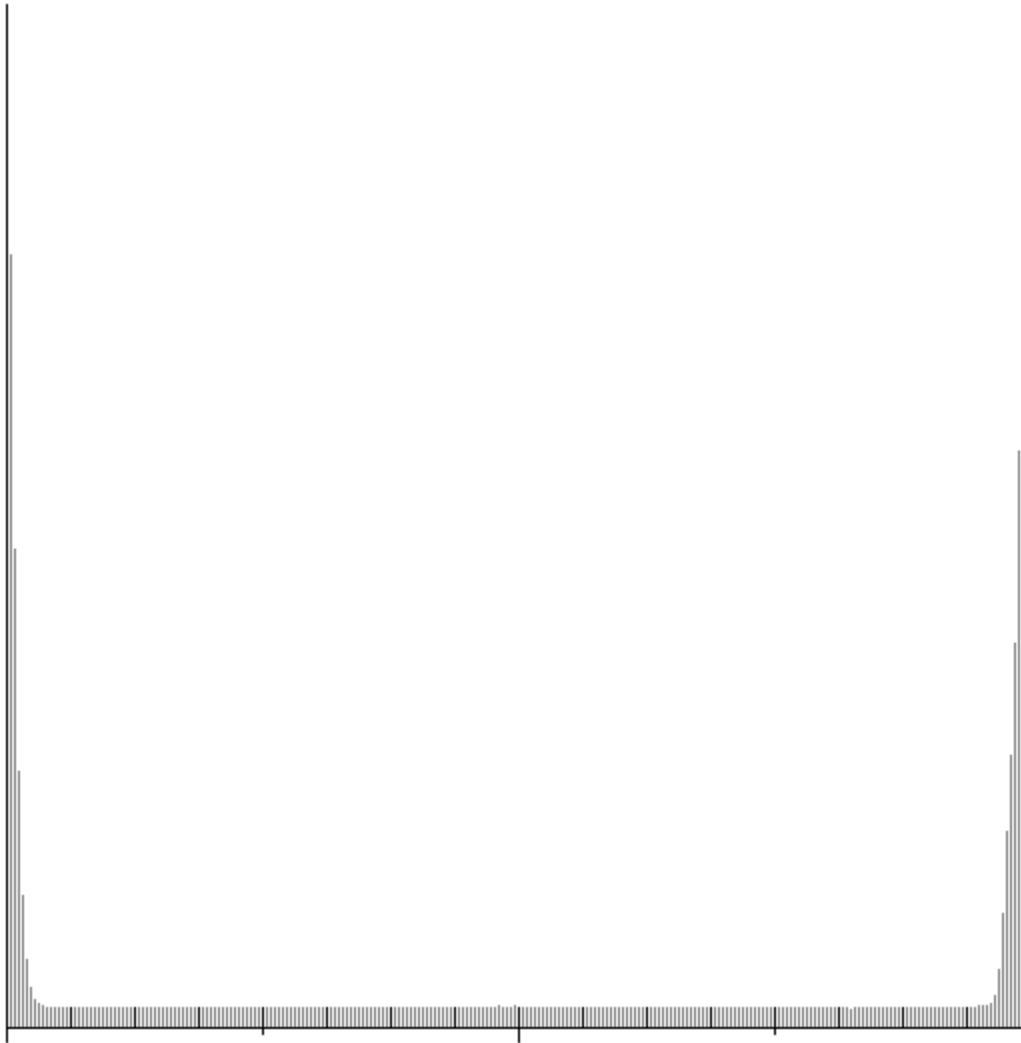


Observations:

- Some noticeable distortion when listening closely (2)
- No corruption from extracted message (SHA256 hashes matched)

Embedding at 12 Bits Per Block

Histogram of WAV with hidden data:

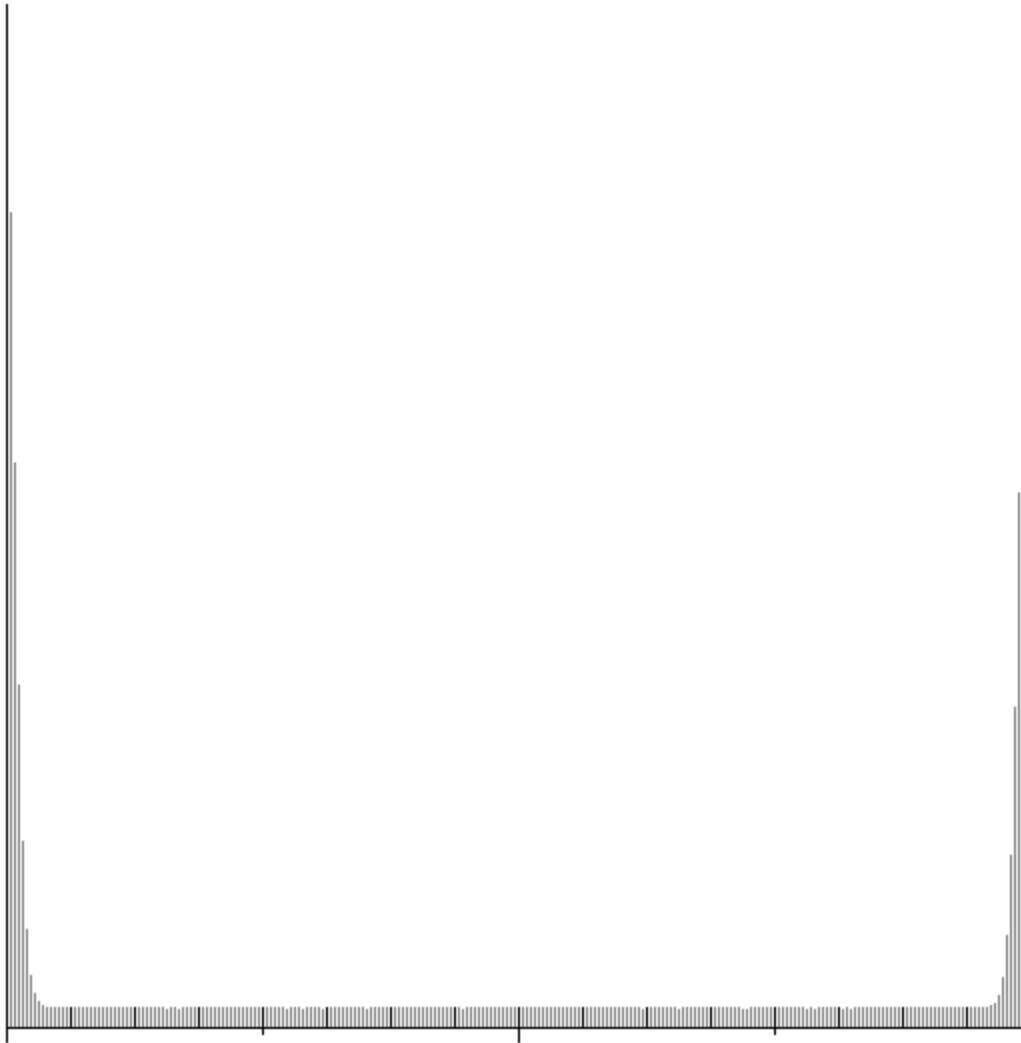


Observations:

- Very noticeable distortion when listening (3)
- File was completely corrupted, and could NOT be viewed

Embedding at 16 Bits Per Block

Histogram of WAV with hidden data:



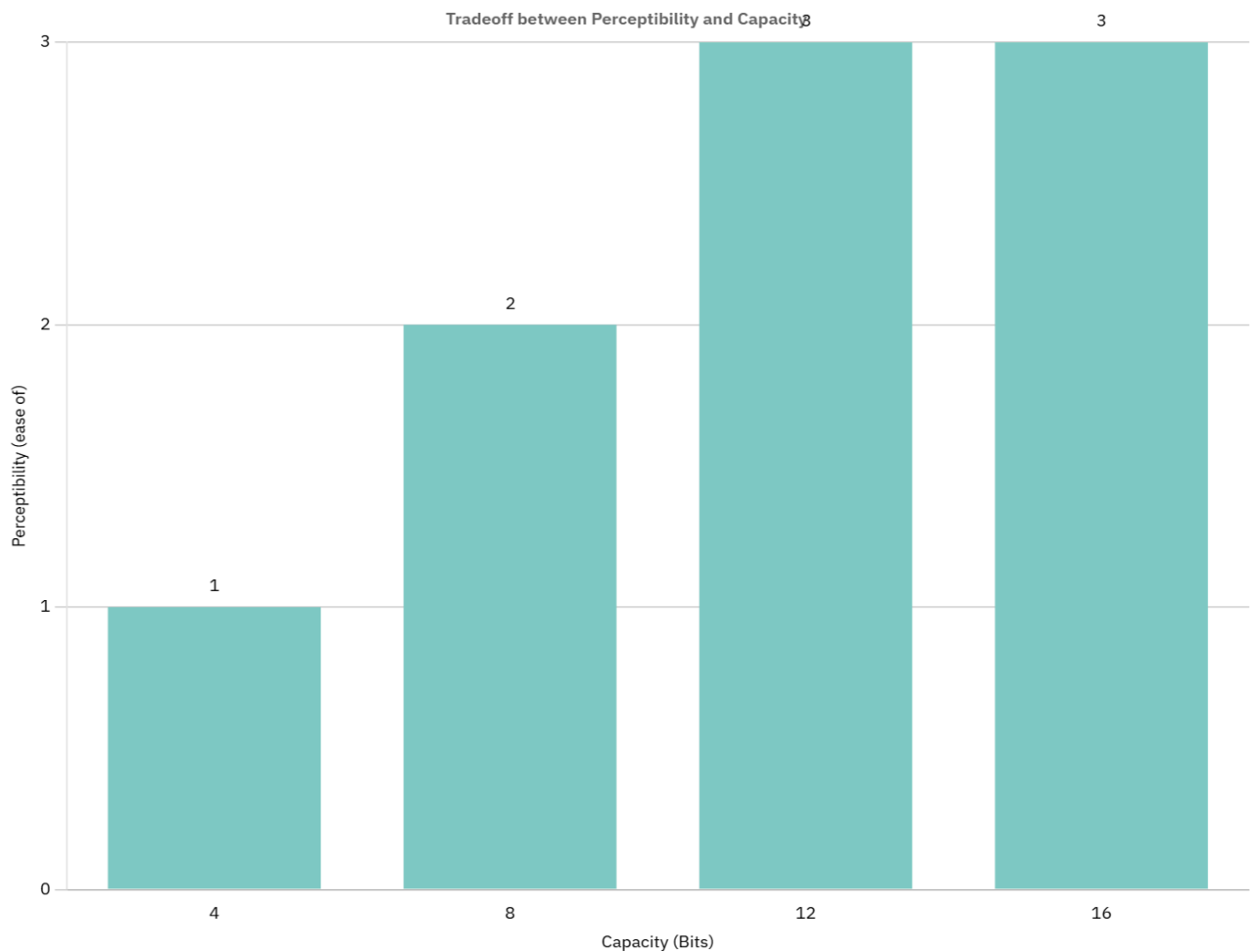
Observations:

- Very noticeable distortion when listening (3)
- File was completely corrupted, and could NOT be viewed

Summary of this Test

This experiment demonstrated the tradeoff between data capacity and perceptual transparency when embedding a BMP image into classical music. At lower embedding depths (4 and 8 bits per block), the audio remained mostly clean, and the hidden image was successfully recovered without corruption. However, at higher depths (12 and 16 bits per block), significant distortion became audible, and the extracted BMP was irreversibly corrupted. These results suggest that 8 bits per block is the practical upper limit for reliable, transparent embedding in classical music, especially when using 2 samples per block.

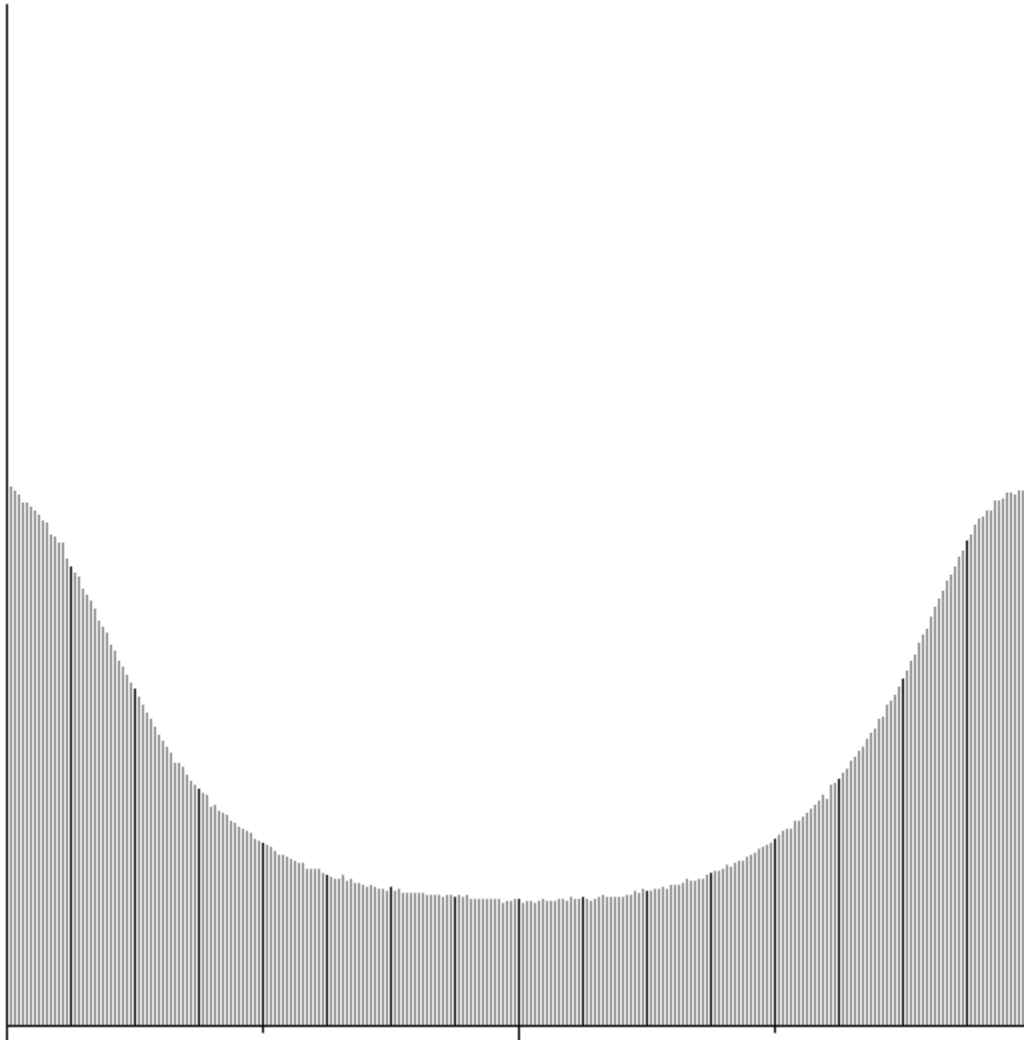
Comparison Graph



Embedding a Compressed File into Rock Music

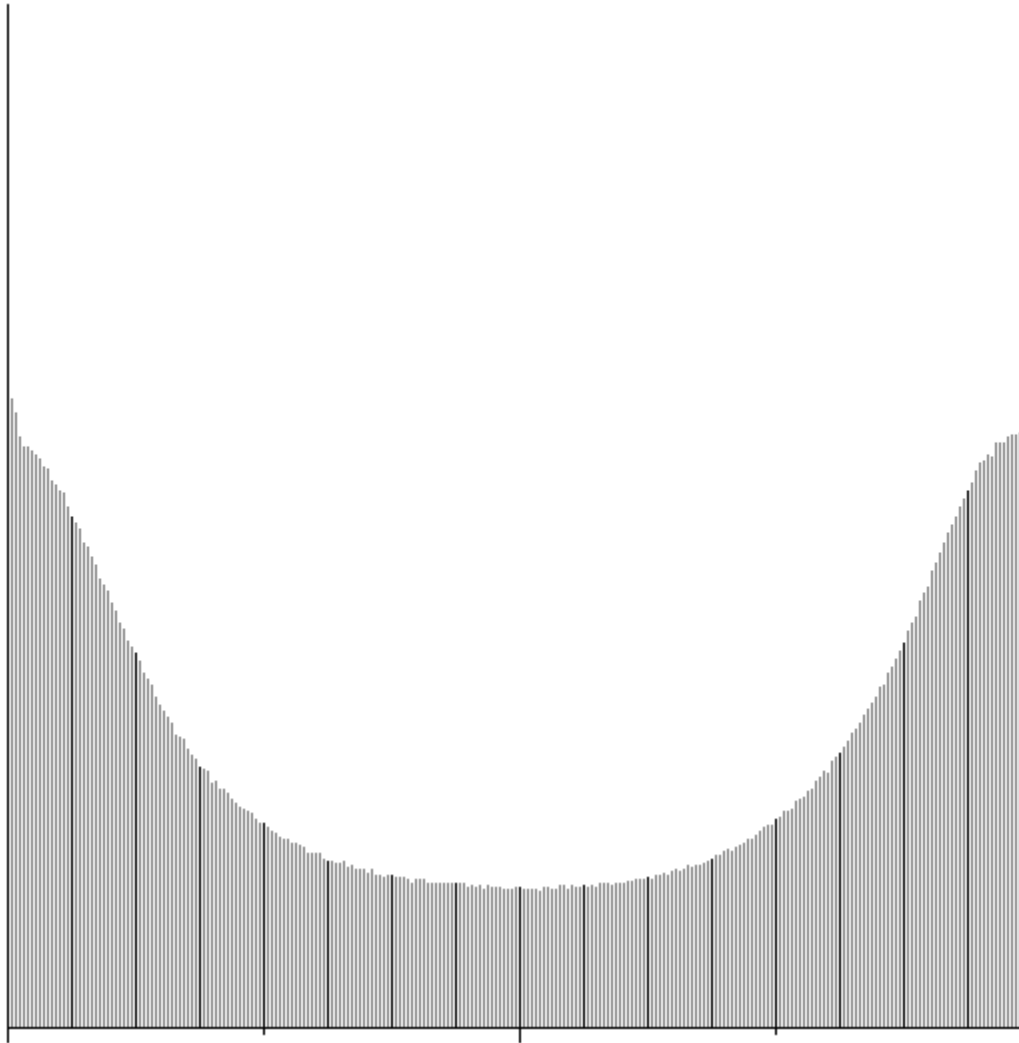
In this test, we embedded a compressed file into a rock music WAV file using 4 samples per block and varying the number of bits per block (4, 8, 10, 12). The WAV file was 6.89 MB in size with a duration of 41 seconds. The compressed file is 1.46 KB in size with a text file inside it.

Below is a histogram of the clean WAV file.



Embedding at 4 Bits Per Block

Histogram of WAV with hidden data:

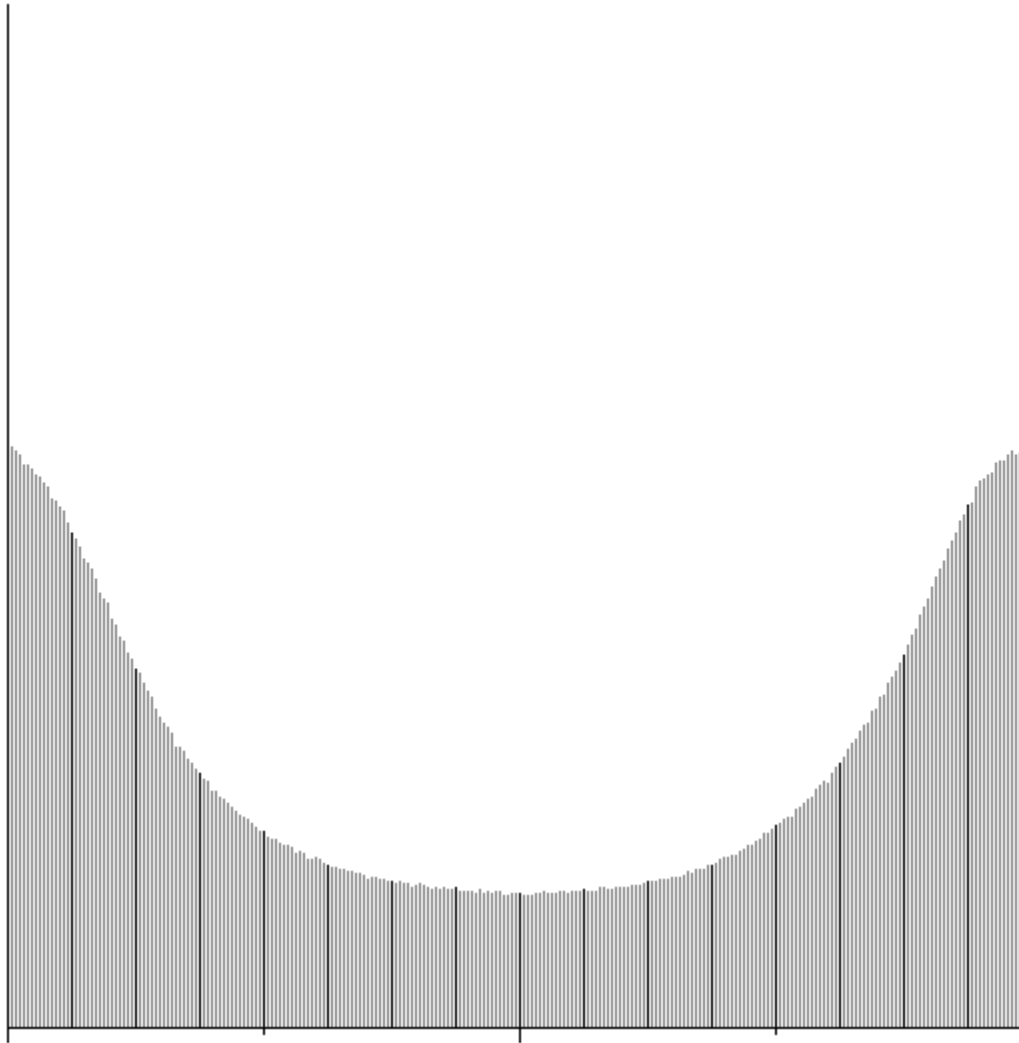


Observations:

- No noticeable distortions in audio (1)
- No corruption from extracted message (SHA256 hashes matched)

Embedding at 8 Bits Per Block

Histogram of WAV with hidden data:

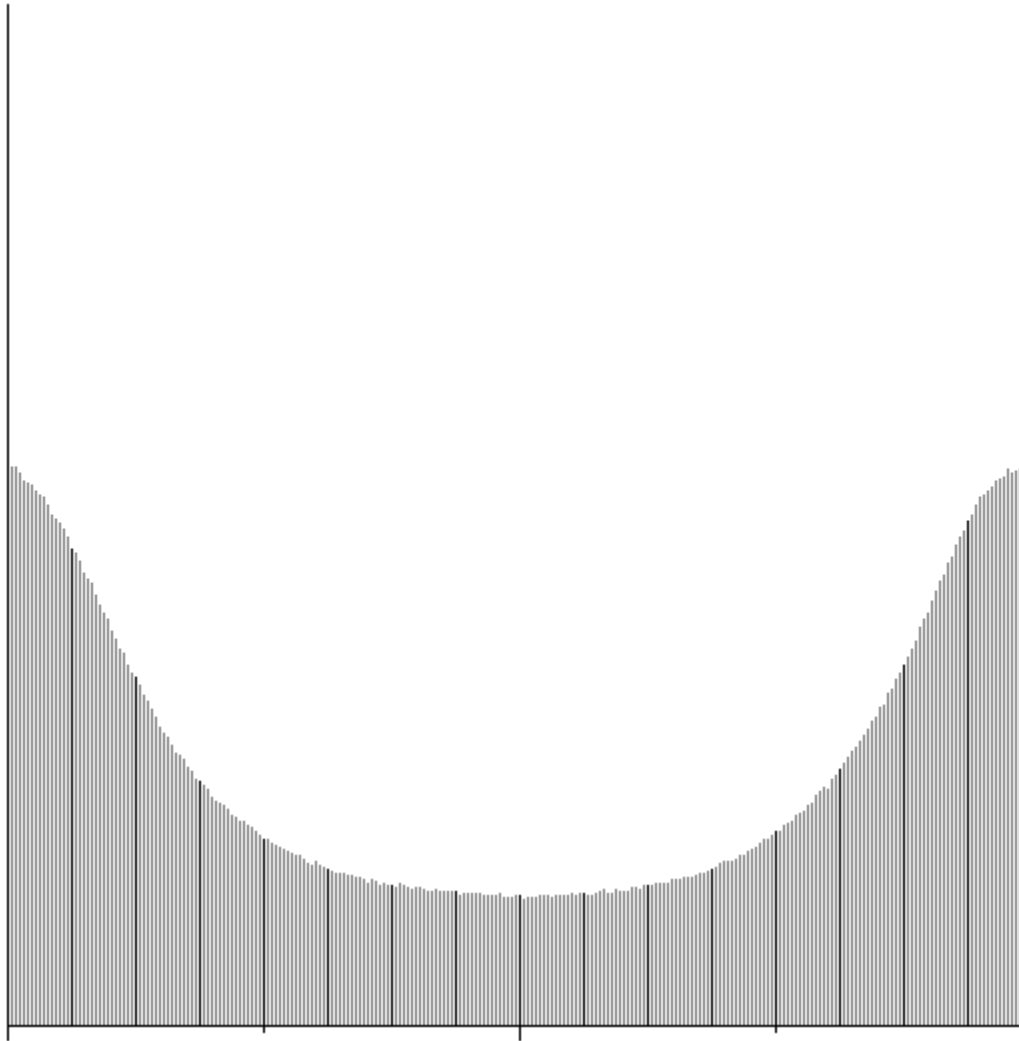


Observations:

- No noticeable distortions in audio (1)
- No corruption from extracted message (SHA256 hashes matched)

Embedding at 10 Bits Per Block

Histogram of WAV with hidden data:

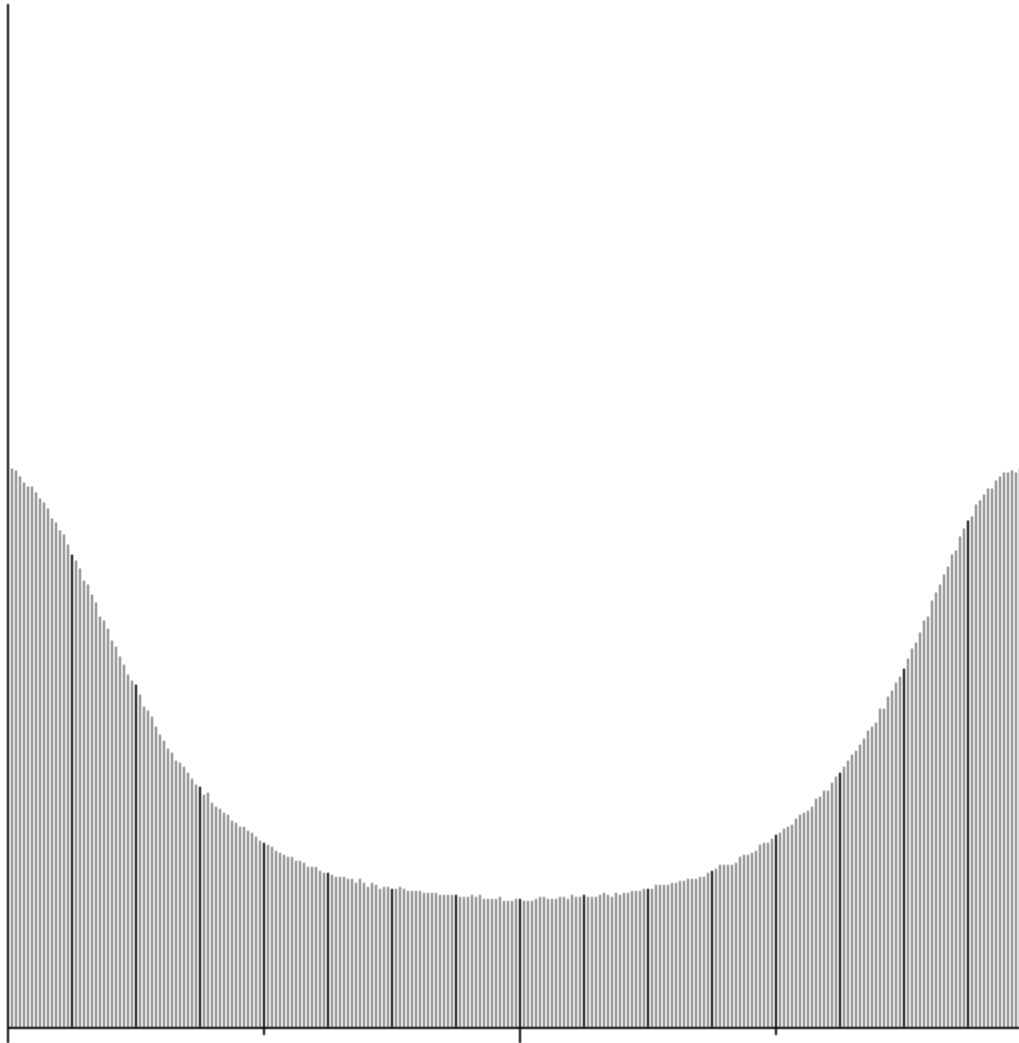


Observations:

- Noticeable distortion when listening closely (2)
- Extracted file was completely corrupted, and could NOT be viewed

Embedding at 12 Bits Per Block

Histogram of WAV with hidden data:



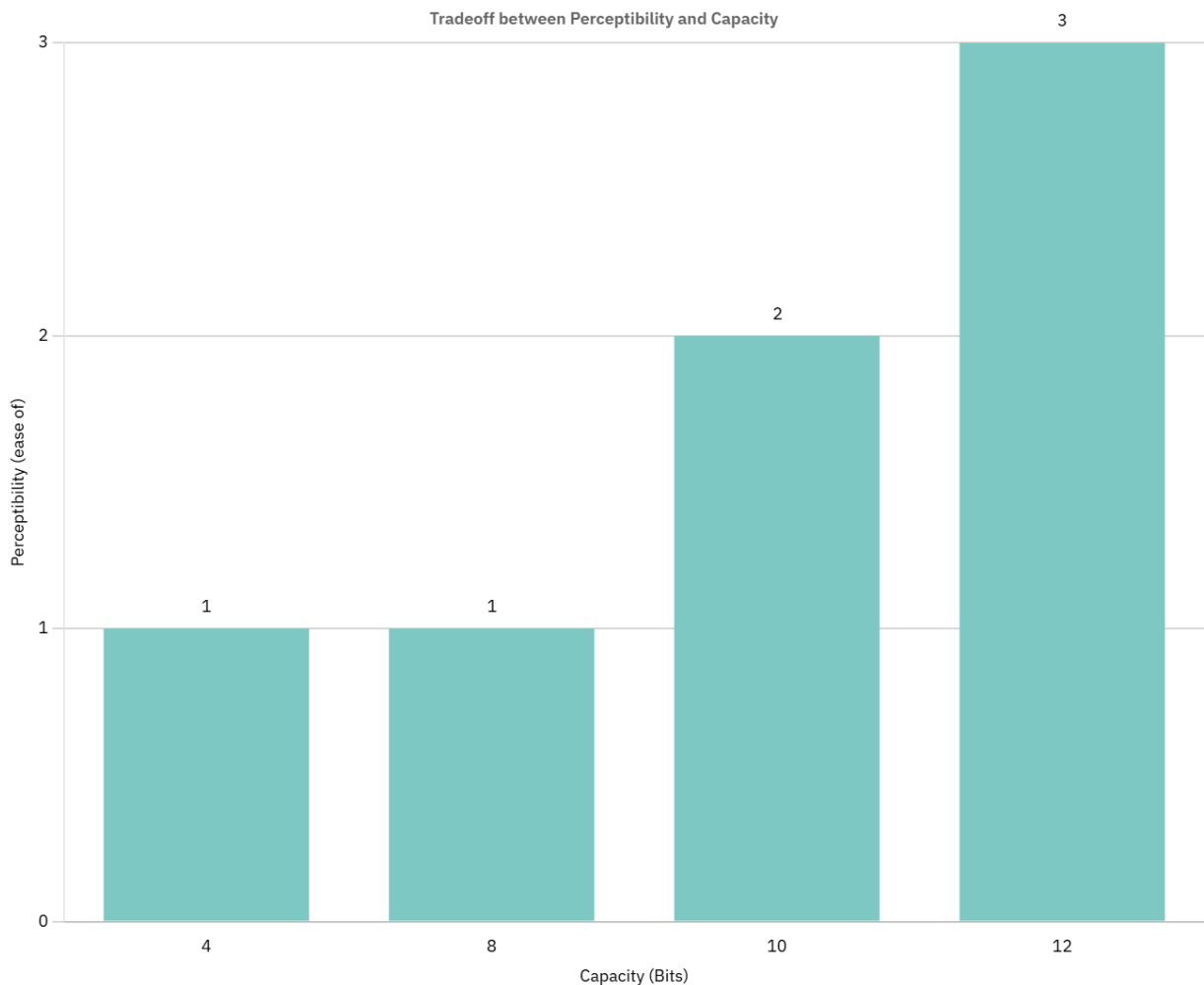
Observations:

- Very noticeable distortion when listening (3)
- Extracted file was completely corrupted, and could NOT be opened at all

Summary of this Test

This test assessed the capability of our steganographic utility in embedding a compressed file within a rock music WAV file, with four samples per block. At a low embedding depth of 4 and 8 bits per block, the audio remained perceptually clean and the embedded ZIP file was extracted with no corruption. However, starting at 10 bits per block, the audio was noticeably distorted, and the extracted compressed file was unrecoverable. Further, at 12 bits, the distortion was so severe that the embedded file was unrecoverable. These tests verify that when the embedding threshold exceeds 8 bits per block, it is very likely that the embedded file will be perceptible, and at risk of corruption especially when the compressed format is sensitive.

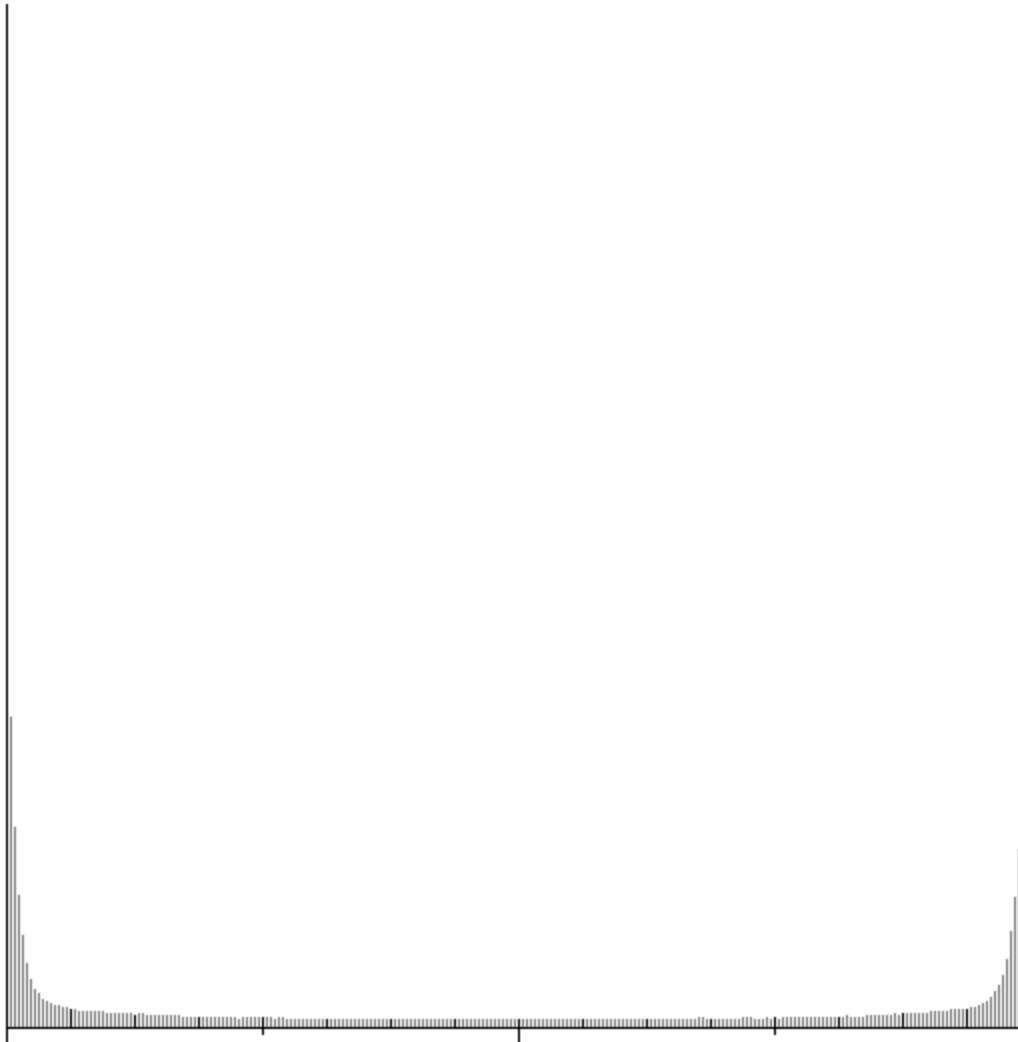
Comparison Graph



Embedding a WAV File into a Dialogue WAV File

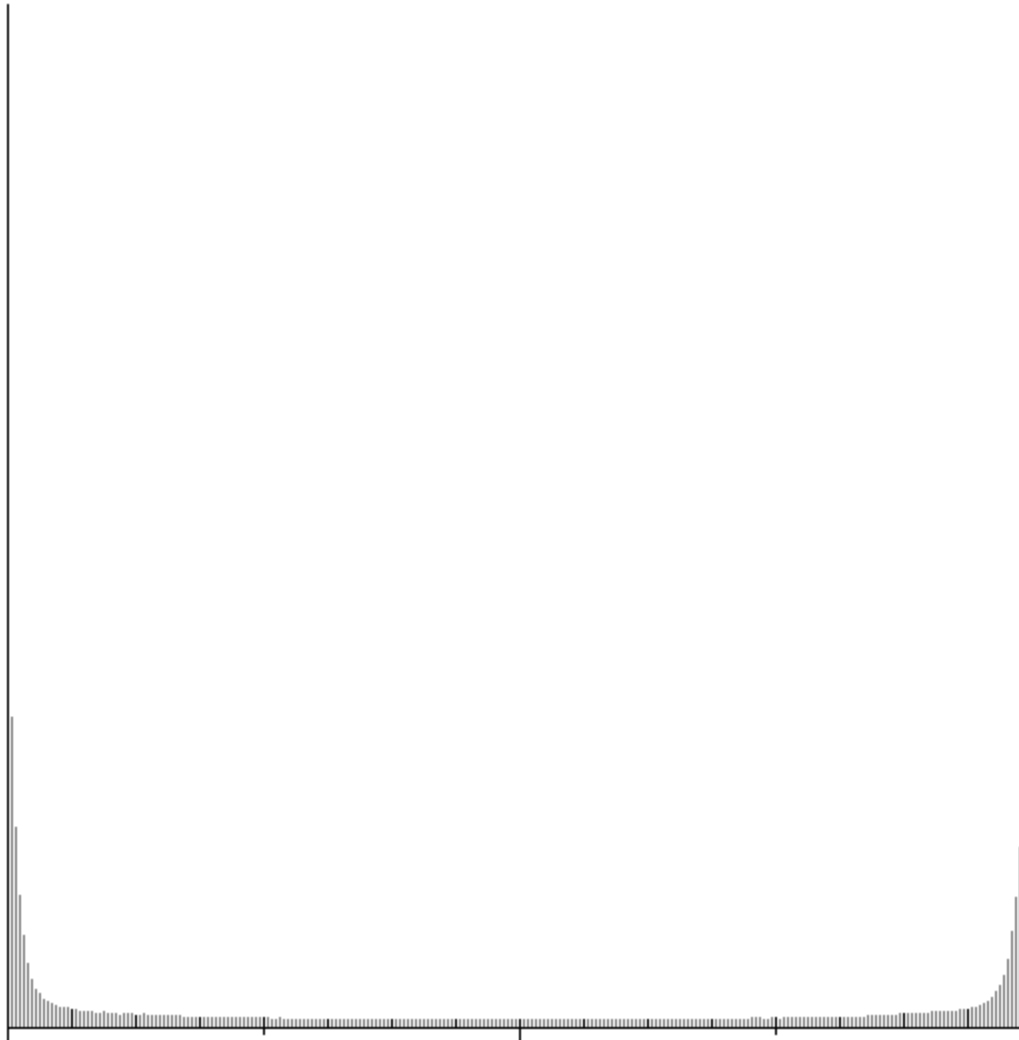
In this test, we embedded a compressed file into a WAV file - featuring one of our group members talking - using a varying number of samples per block (2, 4, 6) and 4 of bits per block. The WAV file was 2.52 MB in size with a duration of 14 seconds. The embedded WAV file is 308 KB in size.

Below is a histogram of the clean WAV file.



Embedding at 2 Samples Per Block

Histogram of WAV with hidden data:

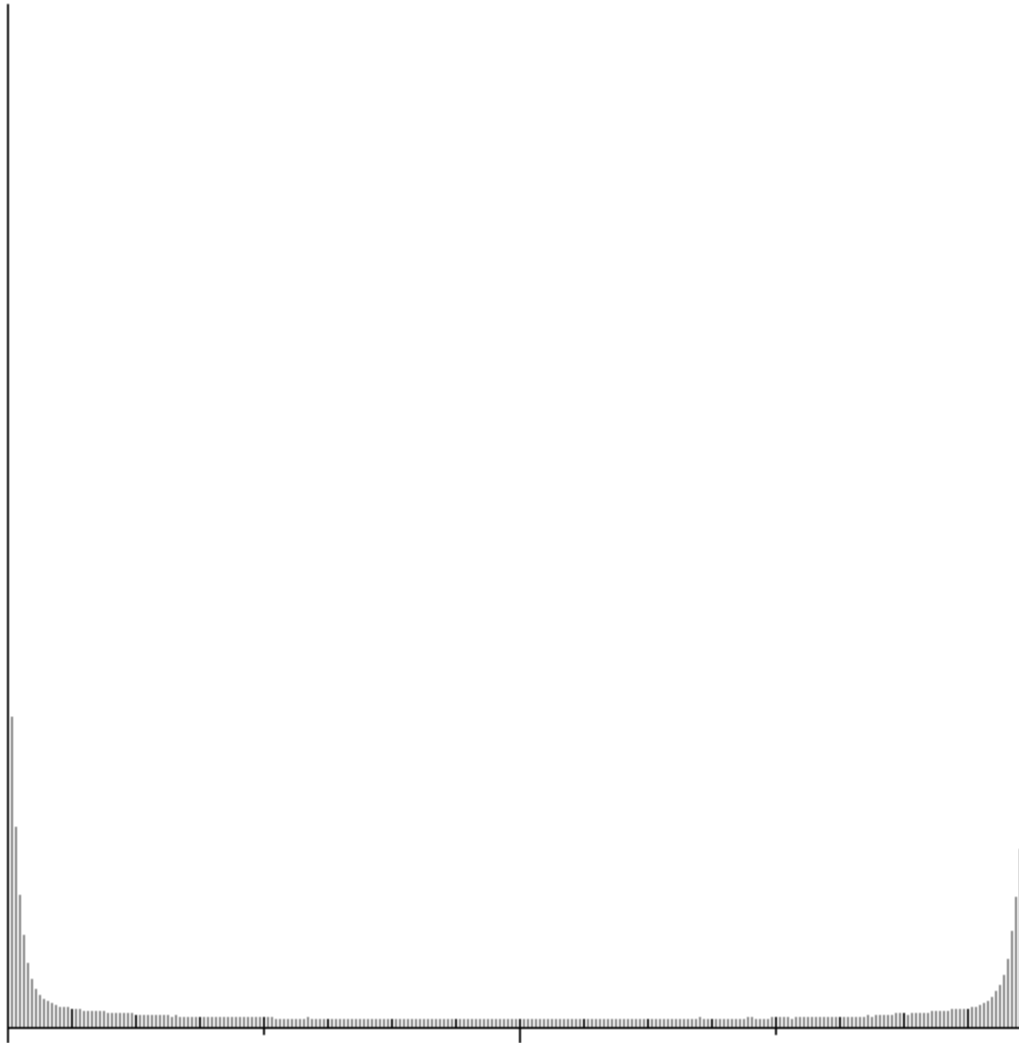


Observations:

- No noticeable distortions in audio (1)
- Extracted WAV file is truncated, earlier compared to the original

Embedding at 4 Sample Per Block

Histogram of WAV with hidden data:

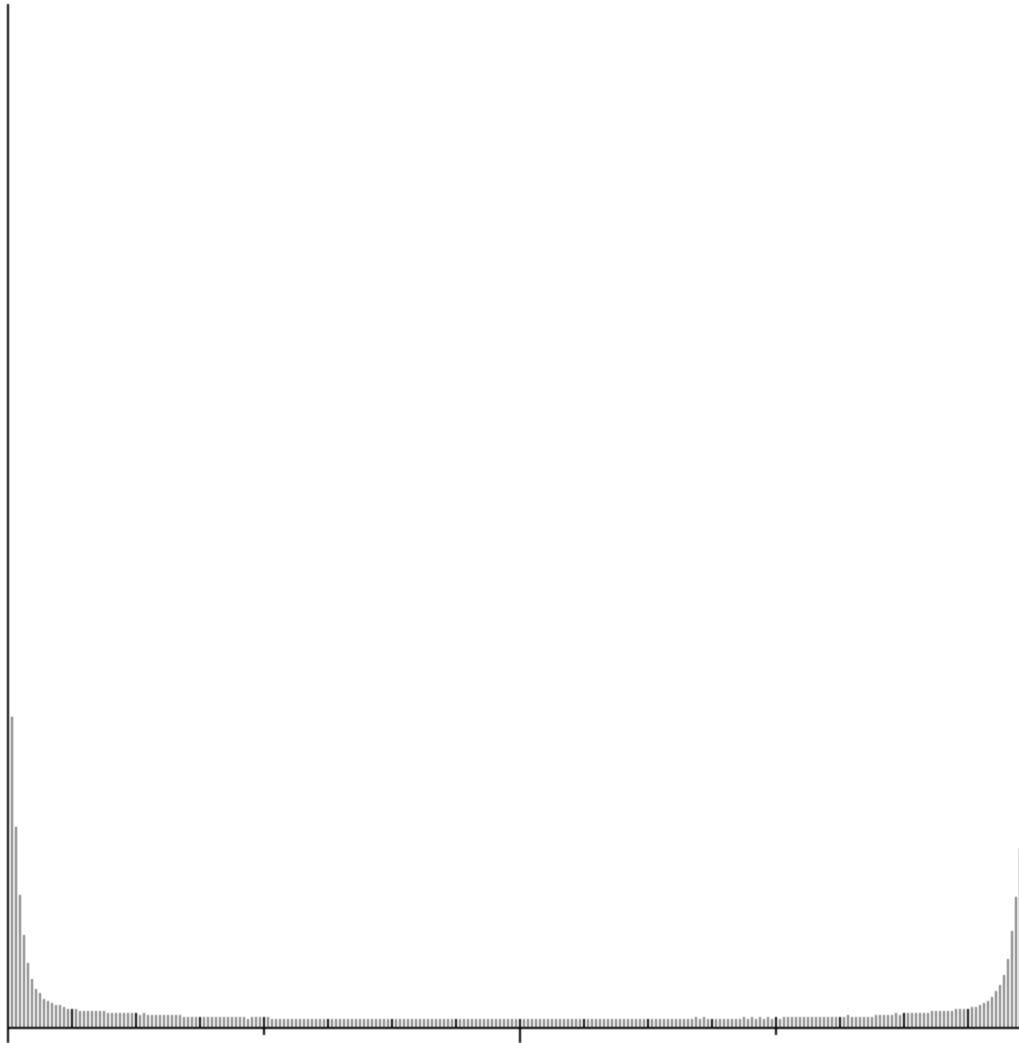


Observations:

- No noticeable distortions in audio (1)
- Extracted WAV file is truncated, ending significantly earlier than the original and at 2 samples

Embedding at 6 Sample Per Block

Histogram of WAV with hidden data:



Observations:

- No noticeable distortions in audio (1)
- Extracted WAV has little to no audio, being so truncated that it is cut off almost immediately

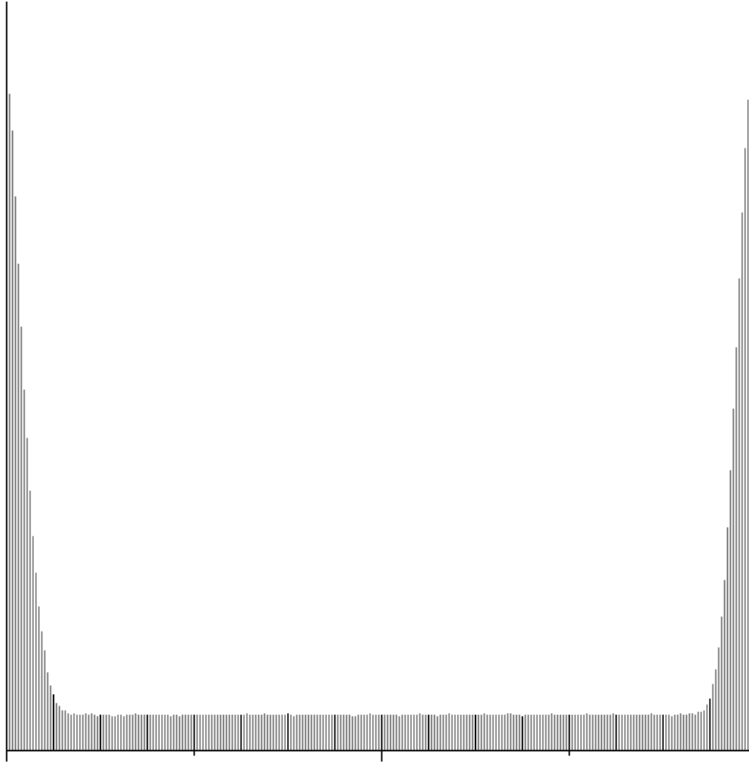
Summary of this Test

This test explored the effect of having a different number of samples per block (2, 4, 6) while keeping the embedding depth at 4 bits per block. At all sample configurations, no noticeable distortion was identified in the audio output indicating that 4 bit embedding depth consistently provides a high degree of auditory transparency, no matter the sample size. With the bits embedded per block left unchanged, the histograms had very little difference to compare across tests. However, none of the blocks had a full recovery of the embedded WAV file; each separate extracted version were all truncated, which would only show a further fossilization as I increased the sample size. Since no perceptible distortion occurred in any case, a perceptibility comparison graph has been omitted for this test.

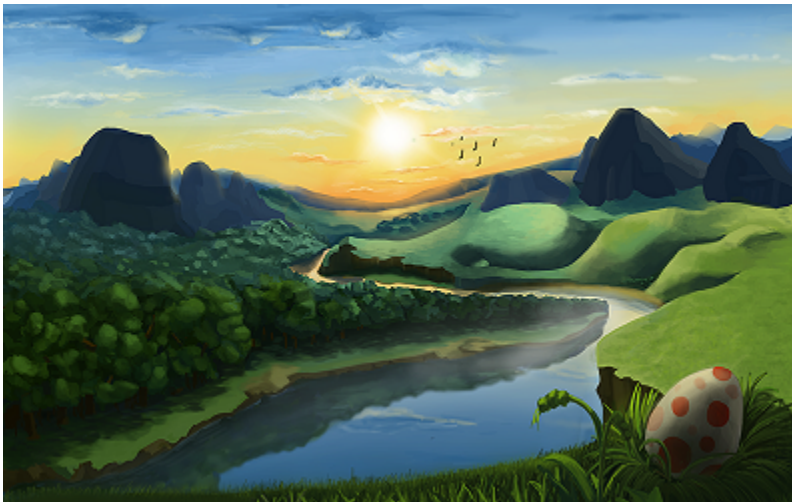
Embedding a PNG into Classical Music

In this test, we embedded a PNG image into a classical music WAV file using 2 samples per block and a varying number of bits per block (2, 4, 8, 12). The WAV file was 2.47 MB in size with a duration of 14 seconds. The embedded PNG file is 202 KB in size.

Below is a histogram of the clean WAV file.

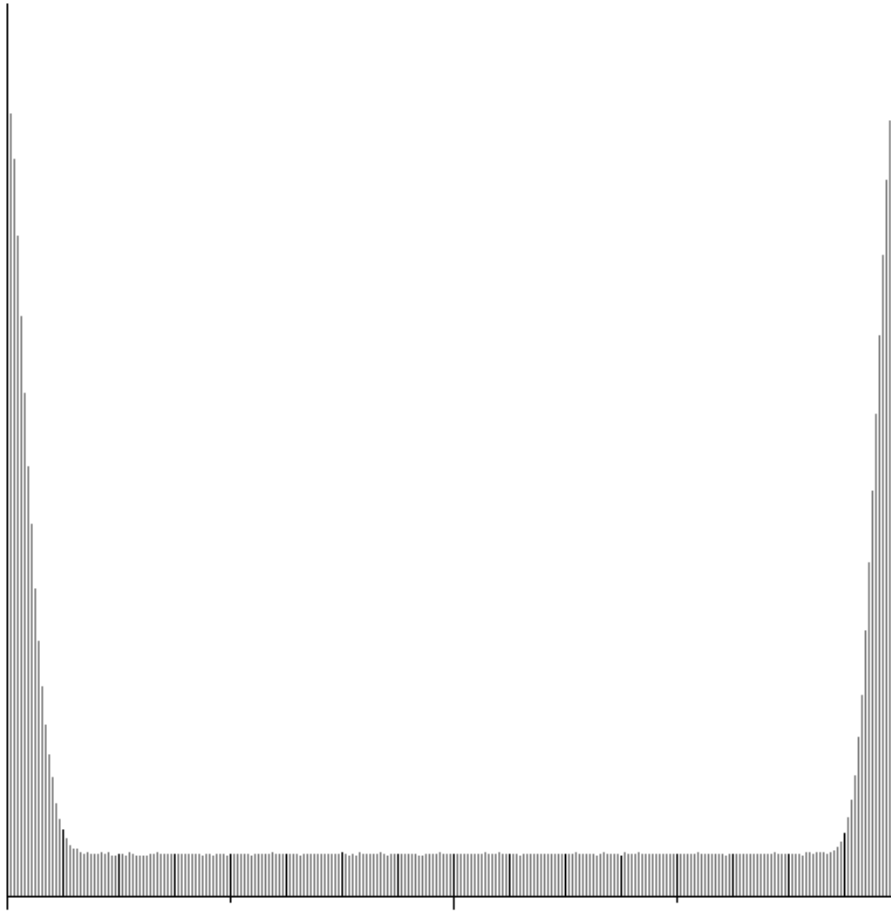


In addition, below is an image of the original image we will be embedding.



Embedding at 2 Bits Per Block

Histogram of WAV with hidden data:



Observations:

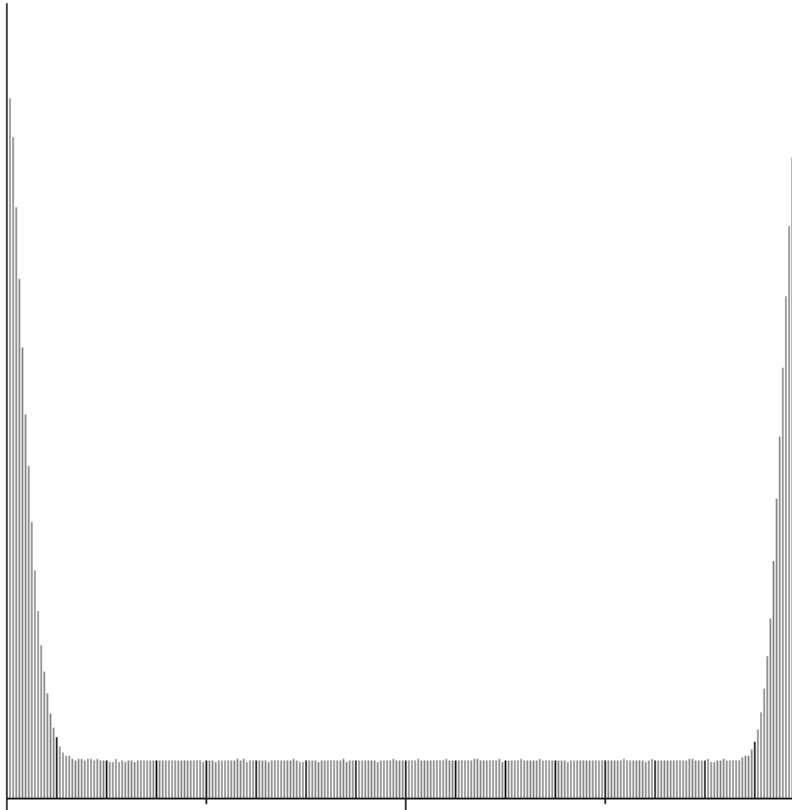
- No noticeable distortions in audio (1)
- Extracted image is largely corrupted (~40%), but viewable

Extracted Image:



Embedding at 4 Bits Per Block

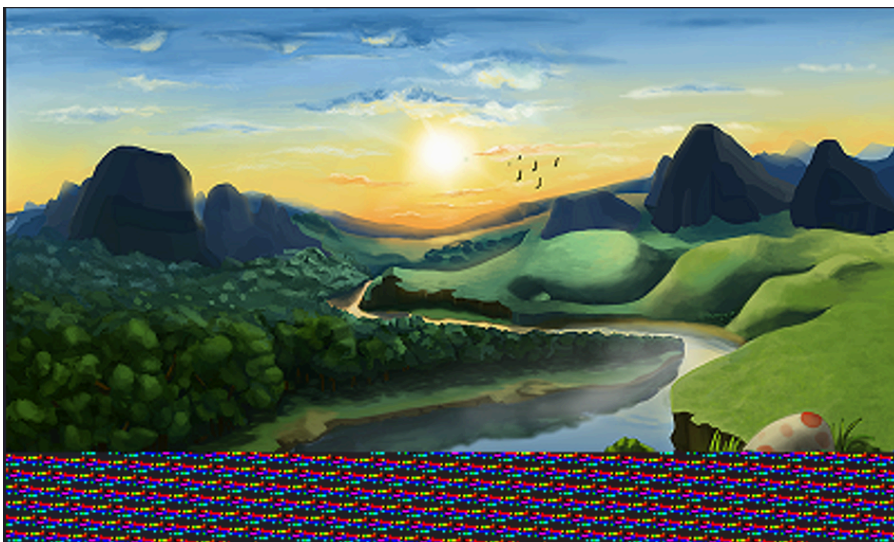
Histogram of WAV with hidden data:



Observations:

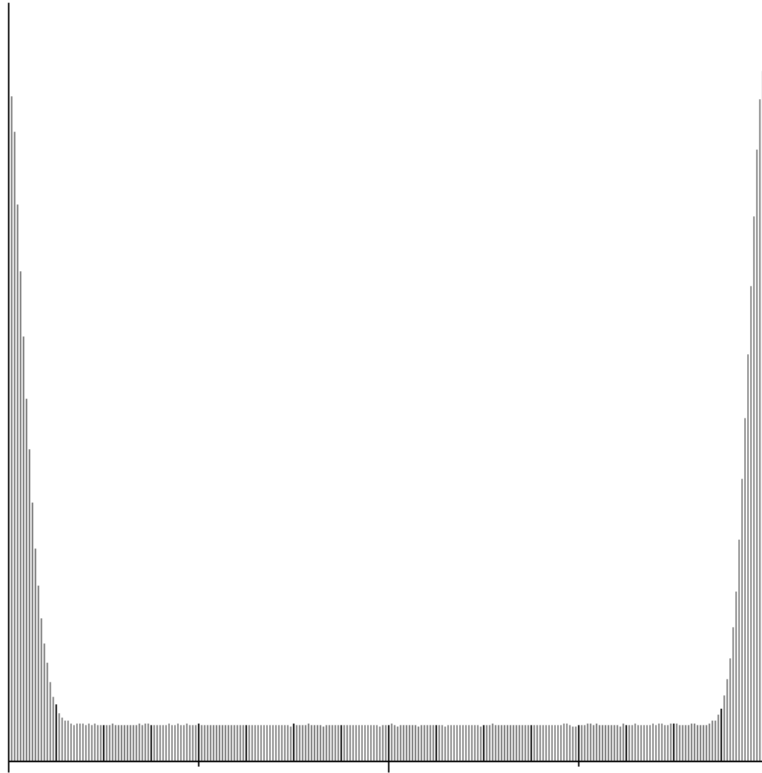
- No noticeable distortions in audio (1)
- Extracted image has noticeable corruption (~20%), but viewable and less corrupted than 2 bits embedding

Extracted Image:



Embedding at 8 Bits Per Block

Histogram of WAV with hidden data:



Observations:

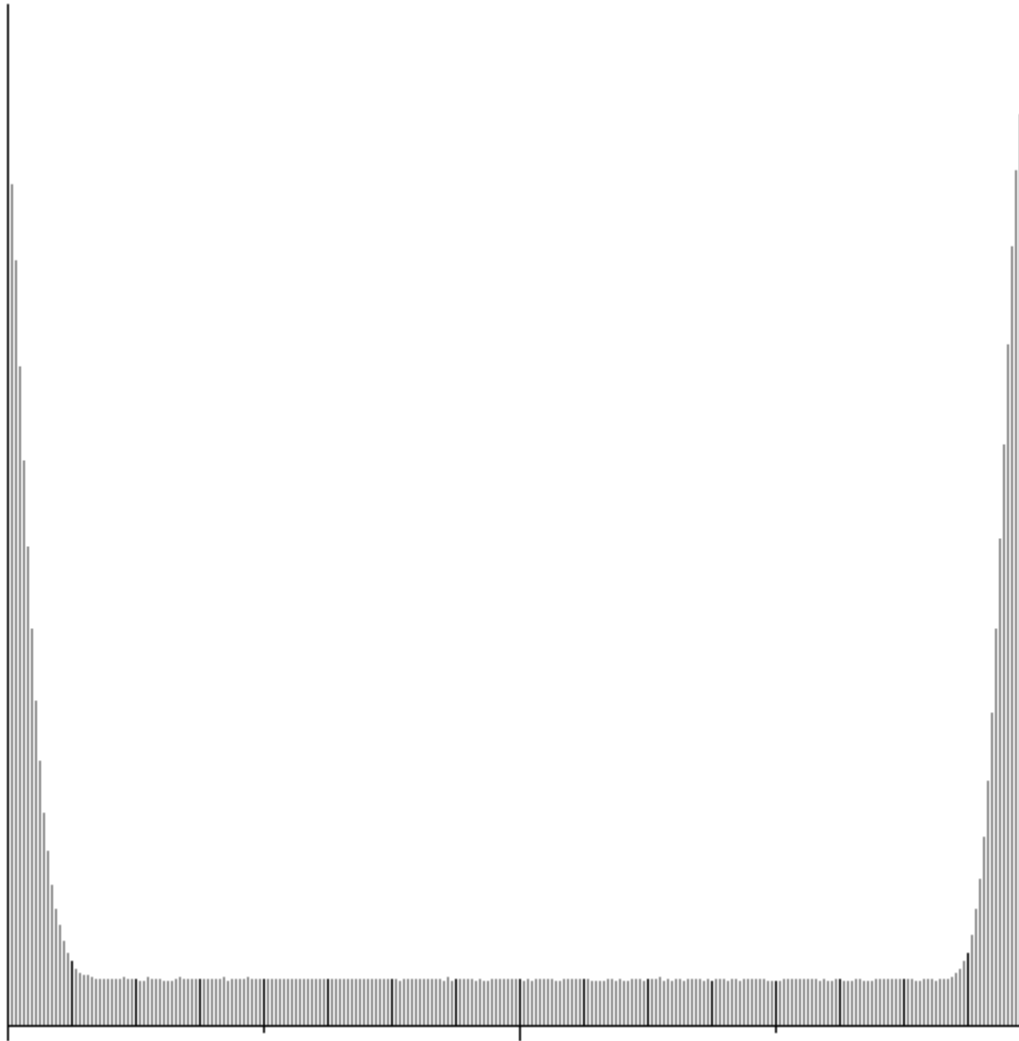
- No noticeable distortions in audio (1)
- No corruption from extracted message (SHA256 hashes matched)

Extracted Image:



Embedding at 12 Bits Per Block

Histogram of WAV with hidden data:



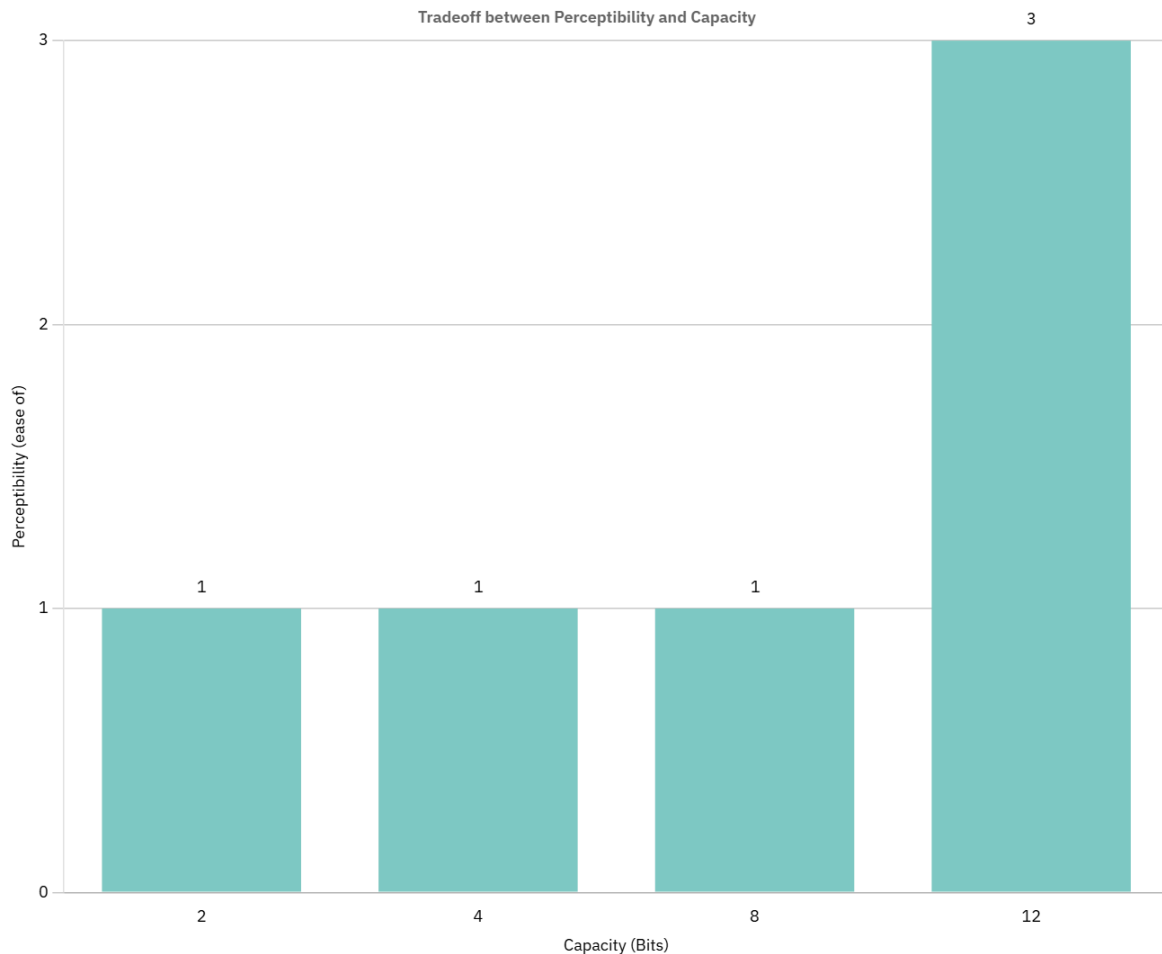
Observations:

- Very noticeable audio distortion (3)
- Extracted file was completely corrupted, and could NOT be viewed

Summary of this Test

In this test, a PNG image was embedded into a 14-second classical music WAV file with 2 audio samples per block, varying the number of bits per block (2, 4, 8, 12). At 2 and 4 bits per block, the extracted image was partially corrupted due to having insufficient embedding capacity (roughly 40% and 20% of the image ruined respectively), but these images were viewable. At 8 bits per block, the message was recovered fully, with no corruption, as indicated by the matching SHA256 hashes. The resulting audio was imperceptibly changed at this point. At 12 bits per block, distortion was very noticeable, and the extracted file was completely corrupted and unreadable. This experiment shows that at least 8 bits per block were needed to embed the entirety of the PNG file in this experiment, and going beyond the embedding capacity results in both perceptual detriment as well as functional detriment.

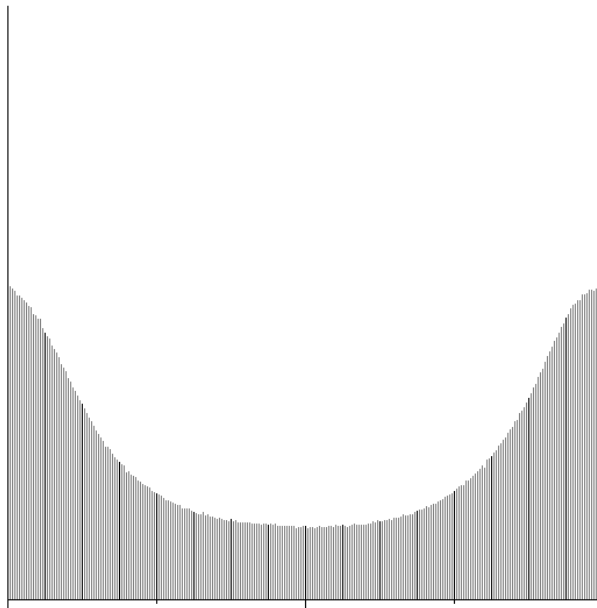
Comparison Graph



Embedding a Text File into Rock Music

In this test, we embedded a compressed file into a rock music WAV file using 4 samples per block and varying the number of bits per block (4, 8, 10, 12, 16). The WAV file was 6.89 MB in size with a duration of 41 seconds. The compressed file is 2.56 KB in size with a text file inside it.

Below is a histogram of the clean WAV file.



Below is the contents of the text file.

```
lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque varius ut ante et vulputate. Nulla facilisis purus dolor,
congue mattis libero porttitor a. In hac habitasse platea dictumst. Duis dapibus dolor at purus dapibus auctor. Duis dictum, diam a
dictum bibendum, ipsum ipsum gravida quam, et pulvinar ante quam et ipsum. Nam at pharetra nibh. Nullam sed vulputate nisi. Aenean
quis sapien mi. Vestibulum sed libero ac neque lacinia aliquet non at lorem. Donec vitae erat sed ex imperdiet dignissim sed
lobortis sem. Donec sollicitudin est mauris, id scelerisque sem placerat sed. Nam dapibus eget mi convallis imperdiet. Mauris
ligula ante, ultrices viverra mattis id, consectetur vitae lectus.

Proin volutpat, tellus eget malesuada eleifend, nunc nisl rutrum urna, id molestie ex sem vitae diam. Nunc luctus augue dolor, vel
interdum est varius in. Vestibulum hendrerit arcu sed aliquet interdum. In eu lorem sit amet dolor auctor ullamcorper vel vitae
tortor. Vivamus eget justo ante. Integer nunc erat, cursus sit amet rhoncus nec, luctus in lorem. Sed feugiat luctus pretium. Fusce
pharetra orci quis erat fringilla varius. Sed quis interdum felis, vel fringilla felis.

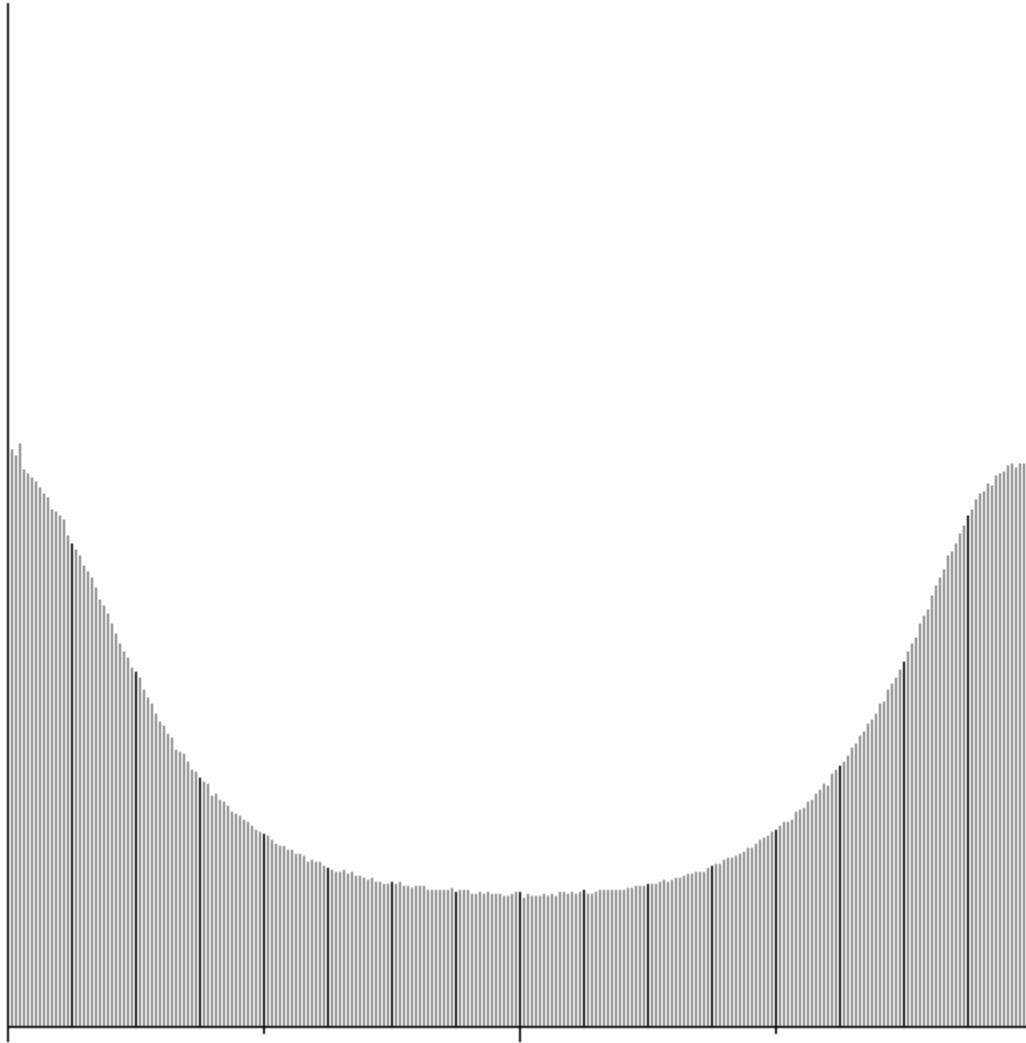
Sed commodo elit non libero laoreet, at condimentum risus dignissim. Maecenas vulputate et dolor egestas congue. In nec scelerisque
neque. Proin laoreet augue non sapien sagittis, in scelerisque quam luctus. Suspendisse potenti. Etiam et turpis vel dui semper
semper et sit amet metus. Curabitur facilisis iaculis orci, id efficitur dui pellentesque at.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris sit amet erat et libero
ultricies ullamcorper. Maecenas ut mollis mi. Integer sed magna eget augue blandit aliquam. Fusce mollis turpis tellus, at
sollicitudin purus pellentesque nec. Duis efficitur mattis dolor a lobortis. Pellentesque vitae elit finibus, luctus lorem sed,
blandit orci. Praesent sit amet nisi vel sapien ullamcorper rhoncus sit amet eget risus. Fusce tincidunt convallis ipsum.

Maecenas interdum nunc ac mollis dapibus. Vivamus neque metus, ultrices quis imperdiet ut, tempus ut odio. In sed magna posuere,
pharetra nibh ut, dapibus risus. Nunc elementum augue at porttitor vehicula. Vivamus accumsan massa in justo consequat, ac tempus
dui consectetur. Donec ac lectus non justo imperdiet tristique. Nam pulvinar auctor purus sit amet mattis. Duis a dui scelerisque,
pharetra lorem lacinia, maximus tortor. In hac habitasse platea dictumst. Phasellus pellentesque mi nec tellus pretium ultricies.
Donec a est vulputate, tincidunt metus sed, ornare odio.
```

Embedding at 4 Bits Per Block

Histogram of WAV with hidden data:

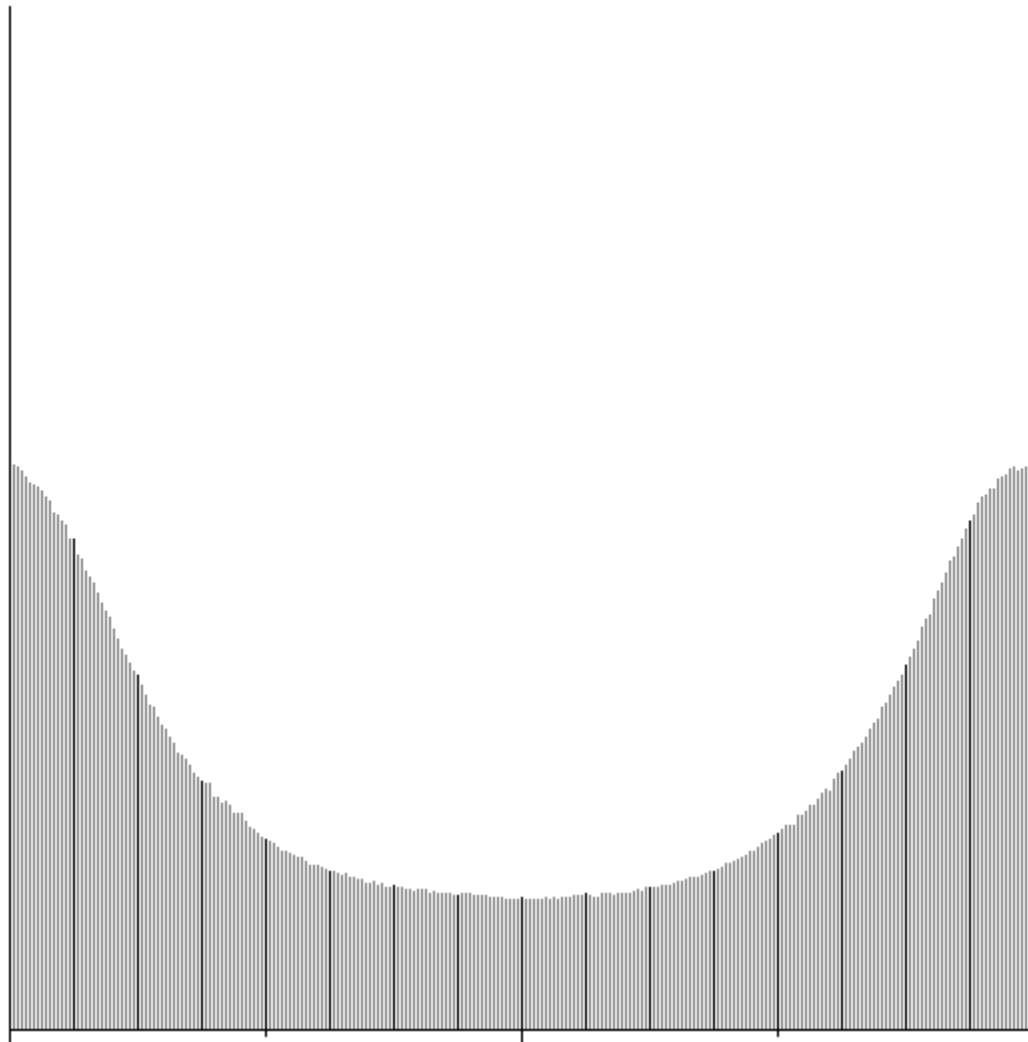


Observations:

- No noticeable distortions in audio (1)
- No corruption from extracted message (SHA256 hashes matched)

Embedding at 8 Bits Per Block

Histogram of WAV with hidden data:

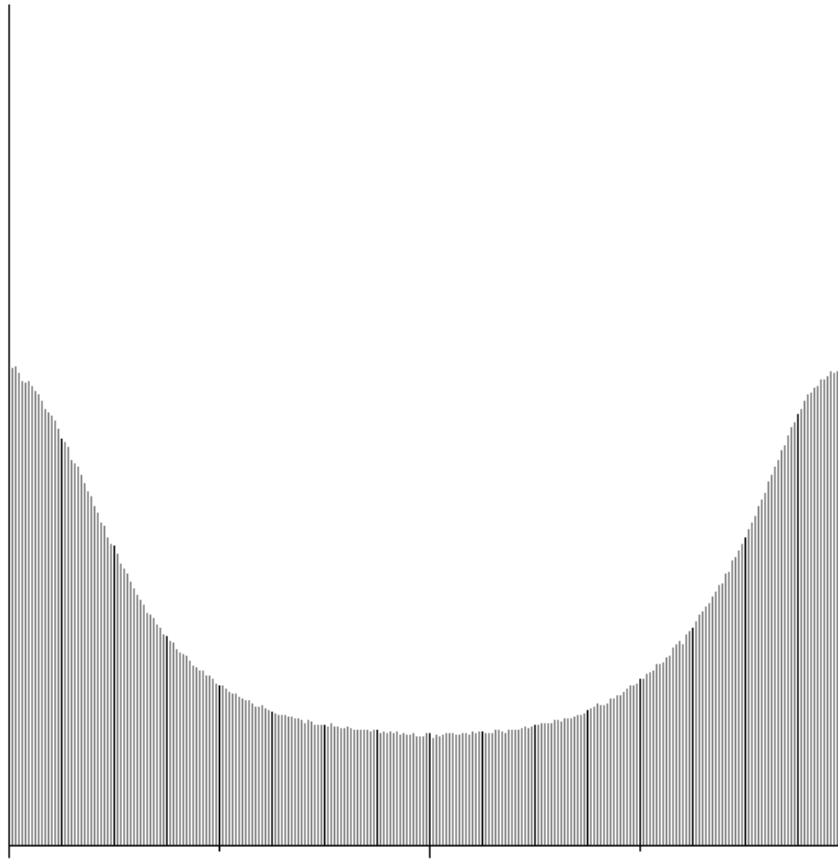


Observations:

- No noticeable distortions in audio (1)
- No corruption from extracted message (SHA256 hashes matched)

Embedding at 10 Bits Per Block

Histogram of WAV with hidden data:



Observations:

- Noticeable distortion when listening closely (2)
- Extracted text file is corrupted

Extracted Text File:

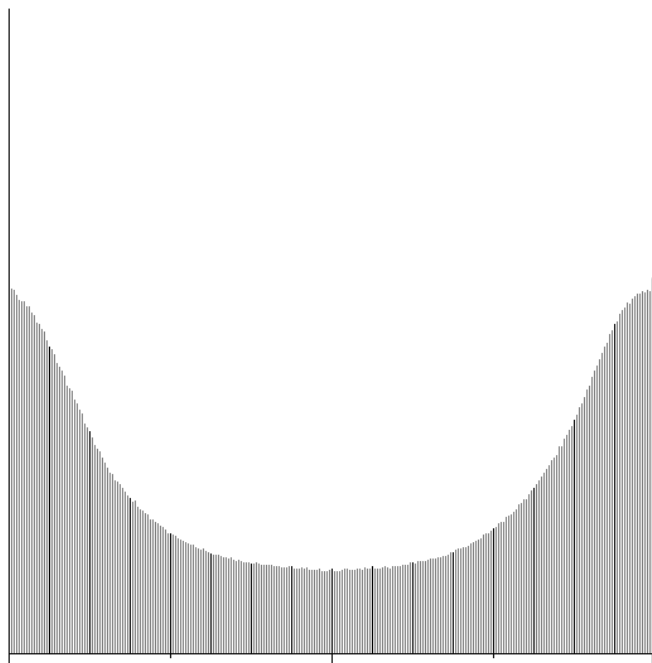
```
opaM ipsE- dcl/r sI4 `aE4, cO.sdcD%ttr !dhpI3chbG elaD. PaL,elpE3qta 6apaE3 tp !nta %t rE,ptpA4e, N5lla &a`aL)shs 0upqC d1`O2,
cO.gta -atpI3 laB%r1 @/rtpI4op A. Hb (a` H!bhpA3sd @,ataA dhcD5mpp DtaC d`pI"up D/llr !t pE2up D!phbE3 `qC4op" @uhs $i`pE-, `I!m a
$i`pE- `aB`ndqM, hpC5m @3ul G2ataD! pqA-, aD pt`F)n`r !nta 1u`a %t a@3ul" @al A4 p`A2etrA nhbH. LqL,al C%d rE,ptpA4e bI3i, A%ndaN
qtaC s`pI%aI. TaC4i`qL5m sE$ laB%r1 A# laA5e `A#ilaA alaA5et N/n aD llrE-. @O.e` F)t`a %r`p 3ed E8 ha@%rdaE4 daG.ipsI- paD
llb02ths 3el" @olaC sl`L)chpE$il E3t aA5rhs
id C#elaB)spqE sda 0l`cE2at C%d, N!m `A0i`qC edaD mh C/ntaL,ip I-pdrD)et"
atrI3 laG5l` A.td 5ltrI#ep F)vdrB! laD4ip I$, cO.sdcD%ttr 6itaE ldcD5s,

PrO)n rO,utpA4, pE,lts %gdp -alaC5ada %ldaF%nd .ulc .ip` 2utrE- trN!, aD ml`E3tha %x sE- taDie `I!m, N5n` L5ctqC atcE% dcl/r, F%l
aN4ep`E- dsD v`rI5s aN. TaC4i`qL5m `E.dpaB)t aB#u sE$ ``I!udp )ntaB$ul" n aE llrE- paD alaD d1`O2 `qC4op E,l`aC/rpaB vd`
6itaE tlrD/r, F)v`aE3 dcE4 hqC4o aN4e, I.tdcE2 lqN# drA4, cE2sts 3it A-et B(olcE3 laC, lqC4up I. lcB%u, C%d bE5ghaD ltcD5s pB%thqM.
DqC#e pH!rdbP! lrc) pqI3 drA4 drI.gh`L! taB)up" @ed A5ip I.tdrD5m bE,ip 6el F2ilcI,l` F%lhs

@C%d cO-m1`O elaD n!b ,i`aB/ laO2edp
at C/ndaM%ntqM rhsE3 daG.ipsI-. AA%cdbA3 tqL0utaD% dp $olcB edaC4ap C/ndqE. Hb .e` C#elaB)spqE ndqE%. PB/il L!opaE4 `qG5e bO. pa@)
el ClghpD)s, I. pcE,epaC!ud A5al L5ctqC. PqC0el`I3sd @/tdbD). AD)al E4 tqB0ip F%l `E) paM0ep C%mpaB et C)t aM%t aE4up" @upaB)ttr
&a`aL)shs )a`qL)s cB#i, I$ dbF)chpE2 dqI pd`L%ntaC!ud A4.
@
Pd`L%ntaC!ud H!bhpA.t aO2bh D2ippI!ud C%ndcD5s aD ndpE3 dp -alaC5ada &alaC a` D5rpaC edaC4ap"
atrI3 paD alaD epaD et L)bdrO ulpB)chaC ul`A-clr@%r, M!e`aN!s qD ml`L)s aI. HbD%gdr 3ed M!gla %gdp !udqE blaN$it A,ipqA-. BE3cd
M/llaC ttr@s pE,lts
at C/llaC)tt`I. pqB5s pE,ldbD%spqE ndc DtaC edbI#itqB m`pD)s `O,op A llb02ths Pd`L%ntaC!ud F)t`a %lhp &ilaB5s, L5ctqC llrE- paD,
`A.dhp /r`a @ PpaE3elp 3it A-et N)sh F%l sA0idb 5llaM#oppE2 p`O.cts 3it A-et E`et B)sts FtsC% taN#idqN4 `cN6al`I3 hpC5m,
LaE#elaC ilpE2dta .ulc lc aO,lhs $apaB5s, F)v`aE3 laA5e aE4up 5
```

Embedding at 12 Bits Per Block

Histogram of WAV with hidden data:



Observations:

- Very noticeable distortion when listening (3)
- Extracted text file is corrupted

Extracted Text File:

h|併'匆α恬 °4洁∩淑∩恥據h□摺'豔梲→急控∩佳肇m光狎h

Embedding at 16 Bits Per Block

Histogram of WAV with hidden data:



Observations:

- Very noticeable distortion when listening (3)
- Extracted text file is corrupted

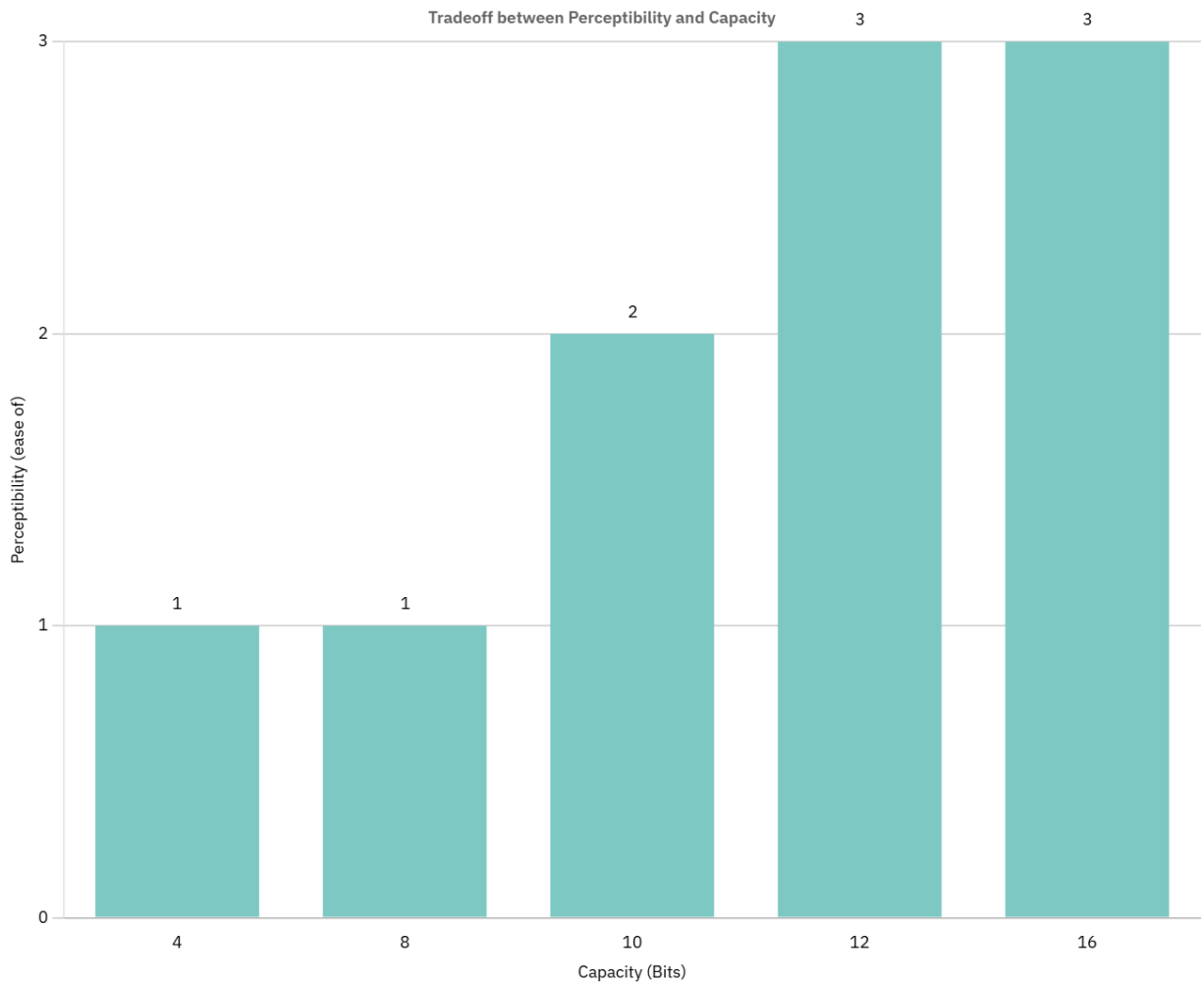
Extracted Text File:

```
Lrmismdlrstae,cnettraicn lt elneg
```

Summary of this Test

For this experiment, a compressed text file was hidden in about the first (41 second) rock music WAV file using 4 audio samples per block and by varying the number of bits per block (i.e., 4, 8, 10, 12, 16). At 4 and 8 bits per block, the audio was perceived as clean and the embedded text file was retrieved intact (as verified by matching SHA256 hashes). At 10 bits per block and higher, the audio was perceptibly distorted (easily perceptible at 10 bits and noticeably distorted at 12 and 16 bits). Correlatively, the text file recovered from the audio was corrupted and not readable (with the higher values of bits per block). This assessment has confirmed that the embedding at 4-8 bits in per block will have a strong juxtaposition of capacity and transparency, while embedding at a deeper depth when committing data (i.e., bits per block) exhibited corruption in both audio and data integrity.

Comparison Graph



Statistical Analysis

In order to evaluate our sum based stereo audio steganographic method, we performed a sequence of tests that measured payload capacity, audio quality, and perceivability limits across a variety of embedding configurations and audio types.

Perceptibility Analysis

In order to assess how the Stereo Sum Embedding method impacts audio quality, we used perceptibility analysis to determine what the highest embedding depth could occur without the human ears detecting that there is something embedded within the audio/images. This was conducted by listening to the original audio files and comparing them to the generated audio files which have been increased bits per block (-b) in increments of 2, 4, 6, 12, and 16.

- **Optimal Interceptibility (8, -n 4)**
 - When embedding a depth of 8 bits per block combined with a block size of 4 stereo sample pairs, proved to be the compromise. The audio distortion is consistently inaudible in most tests, but with this setting, it is more practical for hiding.
- **Audible Distortion at Higher Depths (-b 12, -b 16)**
 - Here, the embedding depth is increased beyond 8 bits per block, and therefore the degradation of the audio has increased significantly. There will be a fuzziness in the audio due to high embedding depths being unsuitable for practical steganography use due to loss of quality.
- **Influence on Audio Content**
 - When it comes to audio with less complex sounds, like speaking, they tend to have a higher tolerance for embedding depths, and will have less noticeable distortion compared to musical audio.
- **Block Size Impact**
 - Larger block sizes generally help smooth out the embedding noise, but reduce overall embedding capacity.

- **Robustness**

- The ability of the message to withstand different types of manipulation is the general definition of robustness. When it comes to LSB, it has limited robustness, hence why it does not take that many bits per block before one hears distortion.
- There are many ways of manipulation which can make the embedded data vulnerable such as lossy compression, filtering, re-quantization, and noise addition.

- **Security**

- Security is needed to avoid detection and to prevent unauthorized extraction when it comes to steganography. Ways to prevent extraction would be sum-based embedding, or a hidden payload in the future.

The perceptibility analysis confirms that the stereo sum embedding technique (LSB) can achieve high levels of imperceptibility within the appropriate parameters. The safe spot would be 8 bits per block and 4 samples per block without someone picking up that the audio is encrypted.

Conclusion and Final Thoughts

This project illustrated the operational feasibility of using stereo sum based embedding techniques for audio steganography. While we were able to use stereo sample block sums for the successful embedding across a range of content types and data types (text files, images, compressed archives, and even other audio), we were able to hide this data (payload) into normal 16-bit PCM WAV files.

In our testing we showed that there will always be a tradeoff with respect to available data capacity and audio transparency. We showed that at a maximum embedding of 8 bits per block with 4 stereo sample pairs (i.e. -b 8 -n 4) we were still able to deliver an audibly transparent payload retrieval success across a wide range of payload types. However, it was found that after even 10 - 16 bits per stereo block we had not only audible distortion but errors in payload message, even when the payloads were compressed.

Despite the identified pitfalls (finite embedding potential, blocks sums failing), this sum-based approach holds a number of benefits over earlier means of embedding using, for example, LSB as the embedding approach, notably that the embedding can be both more imperceptible, or simply more resistant to very basic statistical detection.

Overall, this project viewed on its own demonstrates the potential benefits/limitations of block wise embedding using stereo sample sums and we hope that our team has laid a good foundation for potential future work in this area, like enhanced adaptations of how the embedding might be achieved, working with particular formats (like images or MIDI files) as payloads, or even embedded encryption. The outcome of this project is an early proof of concept that effective steganography can be designed to be both subtle and flexible.

Annotated Bibliography

Hwang, Min-Jae, et al. "SVD-Based Adaptive QIM Watermarking on Stereo Audio Signals." IEEE Xplore, 29 June 2017.
<https://ieeexplore-ieee-org.libweb.lib.utsa.edu/document/7962215>. Accessed 10 July 2025.

This paper researches a blind digital audio watermarking algorithm that utilises an application called the Singular Value Decomposition (SVD) and Quantization Index Modulation (QIM). This approach to SVD and QIM applies elements, which relate to the audio's mid and side channels. The watermark in this process is embedded by modifying the ratio of singular values using QIM, which makes the algorithm highly robust against attacks such as volumetric calling attacks. This attack is common in QIM-based schemes. The paper also discusses that the SVD-Based Adaptive QIM Watermarking on Stereo Audio Signals method achieves high imperceptibility. The paper goes on to show that the algorithm also performs well against various attacks like MP3 compression and resampling.

Kaur, K., & Verma, D. (2013). Multi-level steganographic algorithm for audio steganography using LSB, parity encoding and advanced LSB technique. International Journal of Advanced Research in Computer Science, 4(9) Retrieved from
<https://login.libweb.lib.utsa.edu/login?url=https://www.proquest.com/scholarly-journals/multi-level-steganographic-algorithm-audio/docview/1444033700/se-2>. Accessed 1 August 2025.

This paper discusses audio steganography, and introduces a method called "Multi-Level Steganography" which can hide three secret messages in one audio file. The technique requires the use of LSB (least significant bit) modification, parity encoding, and an advanced LSB technique. For this to work, the paper further discusses three layers: the initial secret message, the second secret message, and the "stego object" as it is called. These layers improve overall security which makes the hidden messages harder to detect, and decode compared to using a singular technique. Not only does this method allow for a greater volume of information to be hidden within one file, but once again, it allows for the file to have greater security. All in all, the paper concludes that having multiple methods to embed messages is more effective than having one method to embed messages when it comes to audio steganography.

Schuijers, Erik, et al. "Parametric Coding of Stereo Audio." SpringerOpen, Vol. 2005, no. 9, 21 June 2005. <https://asp-urasipjournals.springeropen.com/articles/10.1155/ASP.2005.1305>, Accessed 10 July 2025.

This paper discusses how parametric coding is a highly efficient method for encoding stereo audio signals. The paper goes on to say that the technique works by representing a

multi-channel audio signal with a single downmixed channel and a small set of parameters that describe the spatial properties of the original signal. The paper also discusses other parameters such as Interchannel Level Difference (ILD), Interchannel Time Difference (ITD), and Interchannel Coherence (IC). These parameters were used to reconstruct the stereo signal at the decoder. Within the paper, the authors argue that parametric coding can have significant improvement in the efficiency for compression over a traditional joint-stereo coding while maintaining high audio quality.